

PYTHON FOR DATA ENGINEERING

HANDBOOK + PRACTICE SHEET

Part 1: The Python Toolchain Every Data Engineer Needs

Part 2: 45 Interview Questions with Working Code

15 EASY

15 MODERATE

15 ADVANCED

*Generators . Pandas . Polars . PyArrow . DuckDB . PySpark . SQLAlchemy . asyncio . Airflow . Pydantic . pytest
. Packaging . Profiling*

Ingestion	Transformation	Orchestration	Validation
Memory & Perf	Concurrency	Testing	Production

ETL . PY

TABLE OF CONTENTS

PART 1 - PYTHON TOOLCHAIN HANDBOOK

1. Why Python Owns Data Engineering - The Toolchain Map
2. Python Features That Actually Matter for DE
3. Reading & Writing Data - Files, Formats, Compression
4. Pandas for Data Engineering - and Its Hard Limits
5. Polars - The Modern DataFrame (Lazy + Expressions)
6. PyArrow & the Arrow Ecosystem
7. DuckDB - SQL Directly on Files
8. PySpark Essentials for Python Engineers
9. Databases from Python - SQLAlchemy, Drivers, Bulk Loads
10. API Ingestion - requests, httpx, Pagination, Retries
11. Concurrency - Threads vs Processes vs asyncio
12. Airflow with Python - TaskFlow, Idempotency, Backfills
13. Data Validation - Pydantic, pandera, Great Expectations
14. Testing Data Pipelines with pytest
15. Packaging, Config & Secrets
16. Memory & Performance Engineering
17. Production Pipeline Patterns & Repo Structure

PART 2 - 45 PRACTICE QUESTIONS

- Q1-Q15. Easy - Core Python for Data Work
- Q16-Q30. Moderate - Pipelines, Performance & Validation
- Q31-Q45. Advanced - Frameworks, Scale & Production Design

APPENDIX

- A. Common Traps & Interview Tips
- B. Library Selection Cheat Sheet

PART 1

THE PYTHON TOOLCHAIN FOR DATA ENGINEERING

1 - WHY PYTHON OWNS DATA ENGINEERING

Python is not the fastest language, and that is precisely why it works. In data engineering, Python is the **glue and control plane**: it orchestrates work, describes transformations, and calls into highly optimized native engines (Arrow, Spark JVM, DuckDB, NumPy) that do the actual number crunching in C++ or Rust. Your Python code should almost never be in the hot loop. When it is, that is usually the bug.

The Toolchain Map

Lifecycle Stage	Primary Libraries	What They Do
Ingestion	requests, httpx, boto3, kafka-python, confluent-kafka	Pull from APIs, object storage, streams. httpx for async, confluent-kafka for high throughput.
File I/O	pyarrow, fastparquet, csv, orjson, fsspec, s3fs	Read/write Parquet, CSV, JSON. fsspec gives one filesystem API across local/S3/GCS/Azure.
Transformation	polars, pandas, duckdb, pyspark, numpy	In-memory (Polars/Pandas), SQL-on-files (DuckDB), distributed (PySpark).
Databases	SQLAlchemy, psycopyg[binary], snowflake-connector, adbc-driver	Connections, pooling, bulk load. ADBC is the modern Arrow-native alternative to ODBC.
Orchestration	apache-airflow, dagster, prefect	Schedule, retry, express dependencies, backfill, expose SLAs.
Validation	pydantic, pandera, great-expectations, soda-core	Schema and quality enforcement at pipeline boundaries.
Testing	pytest, pytest-fixtures, duckdb, moto, freezegun	Unit-test transformation logic locally without a cluster.
Quality/tooling	ruff, mypy, black, pre-commit, uv	Lint, type-check, format, manage environments fast.
Observability	structlog, opentelemetry, prometheus-client	Structured logs, traces, metrics from pipelines.

TIP: A very common senior interview signal is knowing that you should push work **down** into the engine (a Polars expression, a SQL query, a Spark DataFrame op) rather than looping in Python. 'I would express that as a vectorized operation instead of iterrows' is worth more than any library trivia.

2 - PYTHON FEATURES THAT ACTUALLY MATTER FOR DE

Generators & yield	Stream records lazily with O(1) memory. The single most important Python feature for data engineering - lets you process files larger than RAM.
Context managers	Guarantee cleanup of files, connections, transactions, temp dirs even on exception. Use contextlib.contextmanager for custom ones.
Iterators & itertools	islice, chain, groupby, batched (3.12+) for memory-efficient chunking and streaming composition.
Type hints	Not decoration - they are documentation the IDE and mypy enforce. Critical when pipelines pass dicts and DataFrames between functions.
dataclasses / Pydantic	Model records and configs as typed objects instead of loose dicts. Pydantic adds runtime validation and coercion.

functools	<code>lru_cache</code> for expensive lookups, <code>partial</code> for parameterizing callbacks, wraps for well-behaved decorators.
pathlib	Path objects instead of string concatenation. Cleaner, cross-platform, composable.
Decorators	The idiomatic place for retry, timing, logging, and validation cross-cuts in pipeline code.
<code>concurrent.futures</code>	One API for both thread and process pools. Usually the right level of abstraction for DE parallelism.
Exception chaining	<code>raise X from e</code> preserves the original traceback - essential for debugging multi-stage pipelines.

I Generator pipeline - the core DE Python pattern

```
# The generator pattern: process a 50 GB file with ~constant memory
from pathlib import Path
from typing import Iterator
import csv, qzip

def read_rows(path: Path) -> Iterator[dict]:
    opener = qzip.open if path.suffix == '.qz' else open
    with opener(path, 'rt', newline='', encoding='utf-8') as fh:
        yield from csv.DictReader(fh) # lazy - one row at a time

def clean(rows: Iterator[dict]) -> Iterator[dict]:
    for r in rows:
        if not r.get('order id'):
            continue # drop bad rows
        r['amount'] = float(r['amount'] or 0)
        yield r

def batched(rows: Iterator[dict], size: int = 10_000) -> Iterator[list[dict]]:
    batch: list[dict] = []
    for r in rows:
        batch.append(r)
        if len(batch) >= size:
            yield batch
            batch = []
    if batch:
        yield batch

# Composable pipeline - nothing is materialized until iterated
for chunk in batched(clean(read_rows(Path('orders.csv.qz')))):
    load_to_warehouse(chunk)
```

3 - READING & WRITING DATA

I File I/O patterns

```
import pyarrow as pa
import pyarrow.parquet as pq
import pyarrow.csv as pv
import polars as pl

# ---- Parquet: the default for analytics ----
tbl = pq.read_table('data/orders.parquet',
columns=['order id', 'amount', 'dt'], # column pruning
filters=[('dt', '>=', '2024-01-01')] # predicate pushdown
pq.write_table(tbl, 'out/orders.parquet',
compression='zstd', # better ratio than snappy, still fast
row_group_size=256_000) # tune for skipping granularity
# ---- Partitioned dataset write (Hive-style) ----
pq.write_to_dataset(tbl, root_path='out/orders',
partition_cols=['year', 'month'])
# ---- Streaming Parquet write - never hold it all in RAM ----
schema = pa.schema([('order id', pa.int64()), ('amount', pa.float64())])
with pq.ParquetWriter('out/stream.parquet', schema, compression='zstd') as w:
for batch in record_batches(): # generator of pa.RecordBatch
w.write_batch(batch)
# ---- CSV: fast native readers, not the csv module, for bulk ----
tbl = pv.read_csv('big.csv',
parse_options=pv.ParseOptions(delimiter=','),
convert_options=pv.ConvertOptions(
column_types={'order id': pa.int64(), 'amount': pa.float64()},
null_values=['', 'NULL', 'N/A']))
# ---- fsspec: one API for local / S3 / GCS / Azure ----
df = pl.read_parquet('s3://bucket/orders/*.parquet') # globs work
# ---- JSON: orjson is 2-5x faster than stdlib json ----
import orjson
rec = orjson.loads(raw_bytes) # bytes in, dict out
blob = orjson.dumps(rec) # returns bytes
```

Format	Read With	When To Use	Watch Out
Parquet	pyarrow.parquet / polars	Default for analytics. Columnar, compressed, typed.	Immutable - no row updates. Small files kill performance.
CSV	pyarrow.csv / polars	Interchange, legacy sources.	No types, no compression by default, slow. Use native readers not the csv module for bulk.
JSON/NDJSON	orjson / polars	APIs, semi-structured events.	Verbose, slow to parse. Prefer NDJSON (one object per line) so it streams.
Avro	fastavro	Kafka payloads, schema evolution.	Row-oriented - poor for analytical scans.
Excel	openpyxl / polars	Business inputs (unavoidable).	Slow, type chaos, merged cells. Convert to Parquet immediately.

4 - PANDAS FOR DATA ENGINEERING (AND ITS LIMITS)

Pandas is ubiquitous and fine for medium data, but you must know its failure modes because interviewers probe them. The rule of thumb: pandas needs roughly **5-10x the file size in RAM**, is single-threaded for most operations, and has no query optimizer.

I Pandas: what to avoid, what to do

```
import pandas as pd

# ---- ANTI-PATTERNS (these get you rejected) ----
# for idx, row in df.iterrows(): # ~100x slower than vectorized
# df.loc[idx, 'x'] = row['a'] * 2
# df = df.append(other) # removed in pandas 2.0; was O(n^2)
# df['new'] = df.apply(fn, axis=1) # row-wise Python loop in disguise
# ---- VECTORIZED equivalents ----
df['x'] = df['a'] * 2
df = pd.concat([df, other], ignore_index=True) # concat once, not in a loop
df['tier'] = pd.cut(df['amount'], [0, 100, 500, 1e9],
labels=['low', 'mid', 'high'])
df['flag'] = df['amount'].gt(500) # boolean, vectorized
# ---- Memory control: dtypes are the biggest lever ----
df = pd.read_csv('orders.csv',
dtype={'order_id': 'int32', 'status': 'category', 'amount': 'float32'},
parse_dates=['order_date'],
usecols=['order_id', 'status', 'amount', 'order_date']) # read less
print(df.memory_usage(deep=True).sum() / 1e6, 'MB')
# category can cut a low-cardinality string column by 90%+
df['status'] = df['status'].astype('category')
# ---- Chunked processing for files larger than RAM ----
totals = {}
for chunk in pd.read_csv('huge.csv', chunksize=500_000):
    agg = chunk.groupby('customer_id')['amount'].sum()
    for k, v in agg.items():
        totals[k] = totals.get(k, 0) + v
# ---- pandas 2.x Arrow backend: less memory, faster strings ----
df = pd.read_parquet('orders.parquet', dtype_backend='pyarrow')
```

When Pandas Is the Wrong Tool

Data > ~1/5 of RAM	Use Polars (streaming), DuckDB (out-of-core SQL), or chunked processing.
Need multi-core	Pandas is largely single-threaded. Polars and DuckDB parallelize by default.
Complex multi-step queries	No query optimizer means each step materializes. Polars lazy and DuckDB optimize the whole plan.
Truly distributed scale	Beyond a single large machine, use PySpark. Do not try to shard pandas manually.
Production type safety	Pandas dtypes are loose and silently coerce. Validate with pandera or move to Polars/Arrow typed schemas.

5 - POLARS - THE MODERN DATAFRAME

Polars is a Rust-based DataFrame library built on Arrow with a real **query optimizer** and multi-threaded execution. Its lazy API lets it do predicate pushdown, projection pushdown and operation reordering - the same class of optimization Spark does, but on one machine. For single-node work under a few hundred GB it is typically 5-30x faster than pandas at a fraction of the memory.

I Polars lazy pipeline

```
import polars as pl

# ---- LAZY is the default you should reach for ----
q = (
    pl.scan_parquet('s3://lake/orders/*.parquet') # scan = lazy, read = eager
    .filter(pl.col('order_date') >= pl.date(2024,1,1))
    .filter(pl.col('status') != 'cancelled')
    .with_columns([
        (pl.col('amount') * 1.08).alias('amount with tax'),
        pl.col('order_date').dt.truncate('1mo').alias('month'),
    ])
    .group_by(['month', 'region'])
    .agg([
        pl.col('amount').sum().alias('revenue'),
        pl.col('order_id').n_unique().alias('orders'),
        pl.col('amount').mean().round(2).alias('aov'),
    ])
    .sort(['month', 'revenue'], descending=[False, True])
)

print(q.explain()) # inspect the optimized plan (like Spark's explain)
df = q.collect() # execute

# ---- Streaming engine: process larger-than-RAM ----
df = q.collect(engine='streaming')

# ---- Window functions via .over() ----
df = df.with_columns([
    pl.col('revenue').sum().over('region').alias('region_total'),
    pl.col('revenue').rank(descending=True).over('month').alias('rank_in_month'),
    pl.col('revenue').shift(1).over('region').alias('prev_month'),
])

# ---- Joins and null handling ----
out = orders.join(customers, on='customer_id', how='left')
out = out.with_columns(pl.col('segment').fill_null('unknown'))

# ---- Zero-copy interop ----
arrow_tbl = df.to_arrow() # no copy
pdf = df.to_pandas() # copy unless using arrow dtypes
```

Concept	Pandas	Polars
Execution	Eager, step by step	Lazy by default (scan_*), optimized as a whole plan
Parallelism	Mostly single-threaded	Multi-threaded across all cores automatically
Memory model	NumPy blocks, high overhead	Arrow columnar, far lower overhead
Larger than RAM	Manual chunking	Streaming engine handles it
Missing values	NaN vs None inconsistency	Proper null type, consistent semantics
API style	Positional / label indexing	Expression-based (pl.col(...)), composable and reusable

6 - PYARROW & THE ARROW ECOSYSTEM

Apache Arrow is a language-agnostic **columnar in-memory format**. Its importance is zero-copy interop: Polars, DuckDB, pandas 2.x, Spark and BigQuery can all hand data to each other without serializing. When you see a 10x speedup from 'switching to Arrow', it is usually serialization cost that disappeared.

Table / RecordBatch	Table is the full columnar dataset; RecordBatch is a chunk. Stream RecordBatches to bound memory.
Zero-copy	Handing a Table between Polars, DuckDB and pandas(arrow backend) copies nothing.
Dataset API	pyarrow.dataset treats a directory of Parquet files as one logical table with partition pruning and filter pushdown.
Compute kernels	pyarrow.compute has vectorized C++ operations if you need them without a DataFrame library.
Flight / Flight SQL	High-throughput gRPC protocol for moving Arrow data between systems - far faster than ODBC/JDBC row-by-row.
ADBC	Arrow Database Connectivity: the modern replacement for ODBC/DBAPI that returns Arrow natively, avoiding row-to-column conversion.

I PVArow Dataset API

```
import pyarrow.dataset as ds
import pyarrow.compute as pc

# Treat a partitioned directory as one logical table
dataset = ds.dataset('s3://lake/orders/', format='parquet',
partitioning='hive')
print(dataset.schema)

# Filter + project pushed down to the file scan
tbl = dataset.to_table(
columns=['order id', 'customer id', 'amount'],
filter=(ds.field('year') == 2024) & (ds.field('amount') > 100),
)

# Stream in batches - constant memory over an arbitrarily large dataset
for batch in dataset.to_batches(batch_size=100_000,
filter=ds.field('year') == 2024):
process(batch)

# Zero-copy handoff to DuckDB and Polars
import duckdb, polars as pl
duckdb.sql('SELECT customer_id, SUM(amount) FROM tbl GROUP BY 1').show()
pldf = pl.from_arrow(tbl) # no serialization
```

7 - DUCKDB - SQL DIRECTLY ON FILES

DuckDB is an in-process analytical database - SQLite for OLAP. It reads Parquet, CSV and JSON directly from local disk or object storage, executes vectorized and parallel, and spills to disk when data exceeds RAM. For data engineering it has two killer uses: **ad-hoc and medium-scale transformation without a cluster**, and **unit-testing SQL logic locally**.

I DuckDB for transformation and testing

```
import duckdb

con = duckdb.connect() # in-memory; or connect('warehouse.db')
con.execute("INSTALL https; LOAD https;") # read from S3
con.execute("SET s3 region='us-east-1';")

# Query Parquet on S3 with no ingestion step at all
df = con.execute("""
SELECT date trunc('month', order date) AS month,
region,
SUM(amount) AS revenue,
COUNT(DISTINCT customer id) AS customers
FROM read_parquet('s3://lake/orders/year=2024/*/*.parquet')
```

```
WHERE status <> 'cancelled'
GROUP BY 1, 2
ORDER BY 1, 3 DESC
""").df() # -> pandas; .pl() -> polars; .arrow()
# Mix Python objects and SQL - DuckDB sees local DataFrames by name
import polars as pl
orders = pl.read_parquet('orders.parquet')
customers = pl.read_parquet('customers.parquet')
result = duckdb.sql("""
SELECT c.segment, SUM(o.amount) AS revenue
FROM orders o JOIN customers c USING (customer id)
GROUP BY 1
""").pl()
# Write results straight out
con.execute("COPY (SELECT * FROM result) TO 'out.parquet' (FORMAT PARQUET)")
```

TIP: 'I would prototype and unit-test the transformation in DuckDB, then port the same SQL to the warehouse' is a strong, practical answer. DuckDB in pytest gives you real SQL execution with no infrastructure, which makes pipeline logic genuinely testable.

8 - PYSPARK ESSENTIALS FOR PYTHON ENGINEERS

What PySpark is	A Python API over a JVM engine. Your Python builds a plan; the JVM executes it. Python is only in the hot path if you use Python UDFs.
Why UDFs are slow	A Python UDF serializes each row to a Python worker and back, and blocks Catalyst optimization. Built-in functions run in the JVM.
Pandas UDFs	Vectorized via Arrow - batches of rows as pandas Series. Typically 10-100x faster than a row-wise Python UDF.
Lazy evaluation	Transformations build a DAG; nothing runs until an action (count, write, collect).
Narrow vs wide	Narrow (filter, select, withColumn) = no shuffle. Wide (groupBy, join, distinct, orderBy) = shuffle = stage boundary.
The driver trap	collect() and toPandas() pull everything to the driver. On large data this is an OOM, not a slowdown.
Partition sizing	Target 128-200 MB per partition. spark.sql.shuffle.partitions default of 200 is rarely correct.

I PvSpark production patterns

```

from pyspark.sql import SparkSession, functions as F, Window
from pyspark.sql.types import StructType, StructField, LongType, DoubleType

spark = (SparkSession.builder.appName('orders etl')
        .config('spark.sql.adaptive.enabled', 'true') # always on
        .config('spark.sql.shuffle.partitions', '64') # tune, do not leave 200
        .getOrCreate())

# Explicit schema - never inferSchema in production (reads the file twice)
schema = StructType([
    StructField('order id', LongType(), False),
    StructField('customer id', LongType(), True),
    StructField('amount', DoubleType(), True),
])

df = spark.read.schema(schema).parquet('s3a://lake/bronze/orders/')
# Built-in functions - stays in the JVM, Catalyst can optimize it
out = (df.filter(F.col('amount') > 0)
      .withColumn('amount_with_tax', F.col('amount') * 1.08)
      .withColumn('rn', F.row_number().over(
          Window.partitionBy('customer_id')
                .orderBy(F.col('order_date').desc()))
      .filter(F.col('rn') == 1).drop('rn'))

# Pandas UDF only when a built-in genuinely cannot express it
import pandas as pd
from pyspark.sql.functions import pandas_udf

@pandas_udf(DoubleType())
def normalize(s: pd.Series) -> pd.Series: # vectorized, Arrow-backed
    return (s - s.mean()) / s.std()

# Idempotent partition overwrite - safe to retry and backfill
(out.write.format('delta').mode('overwrite')
 .option('replaceWhere', "dt = '2024-06-01'")
 .save('s3a://lake/silver/orders'))

```

9 - DATABASES FROM PYTHON

I SQLAlchemy: pooling and bulk loading

```
from sqlalchemy import create_engine, text

# Pooling matters: creating a connection per row is a classic bug
engine = create_engine(
    'postgresql+psycopg://user:pw@host:5432/db',
    pool_size=5, max_overflow=10,
    pool_pre_ping=True, # detect dead connections
    pool_recycle=1800, # avoid server-side idle timeouts
)

# Server-side cursor: stream a huge result without loading it all
with engine.connect().execution_options(stream_results=True,
    yield_per=10000) as conn:
    for row in conn.execute(text('SELECT * FROM big_table')):
        handle(row)

# Parameterized queries - never f-string user input into SQL
with engine.begin() as conn: # begin() = transaction + commit
    conn.execute(text('DELETE FROM stg WHERE dt = :dt'), {'dt': '2024-06-01'})

# ---- BULK LOAD: the single biggest performance lever ----
# Postgres COPY is orders of magnitude faster than executemany INSERT
import io, csv, psycopg

def bulk_copy(rows, table: str, columns: list[str], dsn: str):
    buf = io.StringIO()
    writer = csv.writer(buf)
    for r in rows:
        writer.writerow([r[c] for c in columns])
    buf.seek(0)

    with psycopg.connect(dsn) as conn, conn.cursor() as cur:
        cols = ', '.join(columns)
        with cur.copy(f'COPY {table} ({cols}) FROM STDIN WITH CSV') as cp:
            cp.write(buf.read())

# Warehouses: stage to object storage then COPY INTO - not row inserts
# Snowflake: PUT file://... @stage; COPY INTO tbl FROM @stage;
# Redshift: COPY tbl FROM 's3://...' IAM_ROLE '...' FORMAT PARQUET;
# BigQuery: load_table_from_uri('gs://...')
```

The performance hierarchy for loading data is worth memorizing: **row-by-row INSERT** (hundreds of rows/sec) is far worse than **executemany / batch INSERT** (thousands/sec), which is far worse than **COPY or bulk load from staged files** (hundreds of thousands to millions of rows/sec). If an interviewer asks why a load is slow, this is almost always the answer.

10 - API INGESTION

I Resilient API ingestion

```
import httpx, time, random
from typing import Iterator

# ---- Retry with exponential backoff + jitter ----
def request_with_retry(client: httpx.Client, url: str, *,
    max_attempts: int = 5, **kw) -> httpx.Response:
    for attempt in range(max_attempts):
        try:
            r = client.get(url, **kw)
            if r.status_code == 429: # rate limited
                wait = float(r.headers.get('Retry-After', 2 * attempt))
                time.sleep(wait)
                continue
            if 500 <= r.status_code < 600: # transient
                raise httpx.HTTPStatusError('server error',
                    request=r.request, response=r)
            r.raise_for_status() # 4xx = do not retry
```

```
return r
except (httpx.TimeoutException, httpx.HTTPStatusError):
    if attempt == max_attempts - 1:
        raise
    sleep = (2 ** attempt) + random.uniform(0, 1) # jitter
    time.sleep(sleep)
    raise RuntimeError('unreachable')
# ---- Pagination as a generator: memory-safe, composable ----
def paginate(base_url: str, token: str) -> Iterator[dict]:
    with httpx.Client(timeout=30.0,
        headers={'Authorization': f'Bearer {token}'}) as client:
        cursor = None
        while True:
            params = {'limit': 500}
            if cursor:
                params['cursor'] = cursor
            data = request_with_retry(client, base_url, params=params).json()
            yield from data['items']
            cursor = data.get('next cursor')
            if not cursor:
                break
# Incremental: persist the high-water mark so reruns are cheap and idempotent
for rec in paginate(URL, TOKEN):
    if rec['updated at'] <= last_watermark:
        break # sorted desc -> stop early
    buffer.append(rec)
```

11 - CONCURRENCY FOR DATA WORK

Approach	Good For	Why	Watch Out
asyncio	Hundreds/thousands of concurrent HTTP or DB calls	Single thread, no GIL contention, tiny per-task overhead	Needs async libraries throughout (httpx, asyncpg). One blocking call stalls everything.
ThreadPoolExecutor	Tens of concurrent I/O calls, simple code	GIL is released during I/O, so threads genuinely overlap	Useless for CPU-bound work. Thread-safety of shared state.
ProcessPoolExecutor	CPU-bound work: parsing, compression, hashing	Separate processes = separate GILs = real parallelism	Pickling overhead on inputs/outputs. High memory (each process copies).
Native engine parallelism	Almost all dataframe work	Polars/DuckDB/Spark already use all cores in optimized C++/Rust/JVM	Adding Python threads on top usually makes it slower, not faster.

I Bounded asvnc and process pools

```
# ---- asyncio with bounded concurrency (the pattern you want) ----
import asyncio, httpx

async def fetch(client: httpx.AsyncClient, sem: asyncio.Semaphore, url: str):
    async with sem: # cap in-flight requests
        r = await client.get(url, timeout=30.0)
        r.raise_for_status()
        return r.json()

async def fetch_all(urls: list[str], concurrency: int = 20):
    sem = asyncio.Semaphore(concurrency)
    limits = httpx.Limits(max_connections=concurrency)
    async with httpx.AsyncClient(limits=limits) as client:
        tasks = [fetch(client, sem, u) for u in urls]
        return await asyncio.gather(*tasks, return_exceptions=True)

results = asyncio.run(fetch_all(urls))
ok = [r for r in results if not isinstance(r, Exception)]
bad = [r for r in results if isinstance(r, Exception)] # do not lose failures

# ---- Processes for CPU-bound file work ----
from concurrent.futures import ProcessPoolExecutor, as_completed

def parse_file(path: str) -> int: # must be top-level (picklable)
    return heavy_parse(path)

with ProcessPoolExecutor(max_workers=8) as ex:
    futures = {ex.submit(parse_file, p): p for p in paths}
    for fut in as_completed(futures):
        path = futures[fut]
        try:
            print(path, fut.result())
        except Exception as e:
            log.error('failed', path=path, error=str(e))
```

12 - AIRFLOW WITH PYTHON

I Airflow TaskFlow: idempotent and backfill-safe

```
from airflow.decorators import dag, task
from airflow.exceptions import AirflowFailException
from datetime import datetime, timedelta

@dag(
    dag_id='orders daily',
    schedule='0 3 * * *',
    start_date=datetime(2024, 1, 1),
    catchup=True, # enables backfills
    max_active_runs=1, # avoid concurrent writes to same table
    default_args={
```

```

'retries': 3,
'retry delay': timedelta(minutes=5),
'retry exponential backoff': True,
},
tags=['orders', 'silver'],
)
def orders_daily():
@task
def extract(logical_date=None) -> str:
# Use the RUN's date, never date.today() - that breaks backfills
day = logical_date.date()
path = f's3://raw/orders/dt={day}/'
write extract(day, path)
return path
@task
def load(path: str, logical_date=None) -> str:
day = logical_date.date()
# Idempotent: replace only THIS partition
spark_write_replace_where(path, f'dt = '{day}''')
return 'silver.orders'
@task
def quality_gate(table: str, logical_date=None):
day = logical_date.date()
n = row count(table, day)
if n == 0:
raise AirflowFailException(f'No rows for {day} - failing loudly')
nulls = null count(table, day, 'order id')
if nulls:
raise AirflowFailException(f'{nulls} null order id')
quality_gate(load(extract()))
orders_daily()

```

The three Airflow mistakes interviewers look for: using **`datetime.now()`** instead of the run's `logical_date` (breaks backfills silently), doing **heavy work at DAG parse time** (the scheduler re-parses every file constantly, so a database query at module level will cripple it), and **passing large data through XCom** (it goes into the metadata database - pass paths or table names, never DataFrames).

13 - DATA VALIDATION IN PYTHON

I Pvdantic + pandera

```
# ---- Pvdantic: record-level validation and coercion ----
from pvdantic import BaseModel, Field, field_validator, ValidationError
from datetime import date
from typing import Literal

class Order(BaseModel):
    order id: int
    customer id: int
    amount: float = Field(ge=0, description='non-negative')
    status: Literal['new', 'paid', 'shipped', 'cancelled']
    order date: date

    @field_validator('amount')
    @classmethod
    def cap(cls, v: float) -> float:
        if v > 1 000 000:
            raise ValueError('amount exceeds sane bound')
        return round(v, 2)

valid, rejected = [], []
for raw in raw records:
    try:
        valid.append(Order(**raw).model_dump())
    except ValidationError as e:
        rejected.append({'raw': raw, 'errors': e.errors()}) # quarantine

# ---- pandera: DataFrame-level schema contracts ----
import pandera.polars as pa

schema = pa.DataFrameSchema({
    'order id': pa.Column(int, pa.Check.gt(0), unique=True),
    'customer id': pa.Column(int, nullable=False),
    'amount': pa.Column(float, pa.Check.ge(0)),
    'status': pa.Column(str, pa.Check.isin(['new', 'paid', 'shipped'])),
})

validated = schema.validate(df, lazy=True) # lazy: collect ALL errors, not just first
```

The important design decision is **what to do with bad rows**. Failing the whole batch on one bad record is often wrong for ingestion; silently dropping it is always wrong. The correct pattern is quarantine: write rejected records with their error to a dead-letter table, emit a metric, and alert when the rejection rate crosses a threshold. That way nothing is lost and drift is visible.

14 - TESTING DATA PIPELINES WITH PYTEST

I pytest patterns for pipelines

```
# conftest.py - shared fixtures
import pytest, duckdb, polars as pl

@pytest.fixture
def db():
    con = duckdb.connect(':memory:') # real SQL engine, zero infrastructure
    yield con
    con.close()

@pytest.fixture
def sample_orders() -> pl.DataFrame:
    return pl.DataFrame({
        'order id': [1, 2, 3, 4],
        'customer id': [10, 10, 20, 20],
        'amount': [100.0, 250.0, None, 75.0],
        'status': ['paid', 'paid', 'cancelled', 'paid'],
    })

# test_transform.py
```

```

from mvpipeline.transform import revenue by customer
def test_excludes_cancelled(sample_orders):
    out = revenue by customer(sample_orders)
    assert out.filter(pl.col('customer id') == 20)['revenue'][0] == 75.0
def test_handles_nulls(sample_orders):
    out = revenue by customer(sample_orders)
    assert out['revenue'].null_count() == 0 # nulls must not propagate
def test_empty_input_returns_empty_not_error():
    empty = pl.DataFrame(schema={'order id': pl.Int64, 'amount': pl.Float64,
    'customer id': pl.Int64, 'status': pl.Utf8})
    assert revenue by customer(empty).height == 0
@pytest.mark.parametrize('amount,expected_tier', [
    (50, 'low'), (300, 'mid'), (5000, 'high'), (0, 'low'),
    ])
def test_tier_boundaries(amount, expected_tier):
    assert assign_tier(amount) == expected_tier
# SQL logic tested against a real engine
def test_sql_dedup(db, tmp_path):
    db.execute("CREATE TABLE t AS SELECT * FROM VALUES (1,'a'),(1,'b') AS t(id,v)")
    n = db.execute('SELECT COUNT(*) FROM (SELECT DISTINCT id FROM t)').fetchone()[0]
    assert n == 1

```

What to test, in priority order: **transformation logic** on small fixtures (fast, catches most bugs), **edge cases** that production will definitely hit (empty input, all nulls, duplicates, single row, boundary values), **idempotency** (run the load twice, assert the row count is unchanged), and **schema contracts** (the output has exactly the expected columns and types). Do not attempt to test against production data volumes in CI - test logic on fixtures and validate volume separately with data quality checks.

15 - PACKAGING, CONFIG & SECRETS

```
I myproject.toml
# myproject.toml - the single source of truth
[project]
name = 'orders-pipeline'
version = '0.3.0'
requires-python = '>=3.11'
dependencies = [
    'polars>=1.0',
    'pyarrow>=16',
    'pydantic>=2.6',
    'httpx>=0.27',
    'sqlalchemy>=2.0',
]

[project.optional-dependencies]
dev = ['pytest', 'pytest-cov', 'ruff', 'mypy']
[project.scripts]
run-orders = 'orders.pipeline.cli:main' # installs a real CLI entry point
[tool.ruff]
line-length = 100
[tool.mypy]
strict = true
```

```
I Typed configuration with pydantic-settings
# Typed, validated settings from env vars - no scattered os.getenv
from pydantic settings import BaseSettings, SettingsConfigDict
from pydantic import SecretStr

class Settings(BaseSettings):
    model config = SettingsConfigDict(env file='.env', env prefix='PIPE ')
    env: str = 'dev'
    warehouse dsn: SecretStr # never logged accidentally
    s3 bucket: str
    batch size: int = 10 000
    max retries: int = 3
    log level: str = 'INFO'
    settings = Settings() # fails FAST at startup if config is wrong
    dsn = settings.warehouse dsn.get secret value()

# Secrets hierarchy, best to worst:
# 1. Cloud secret manager (AWS Secrets Manager, Azure Key Vault) at runtime
# 2. Injected env vars from the orchestrator's secret backend
# 3. .env file, ignored, local dev only
# X. Hardcoded in source <- never; assume every repo leaks eventually
```

16 - MEMORY & PERFORMANCE ENGINEERING

Stream, do not load	Generators and Arrow RecordBatches keep memory flat regardless of input size. This is the first tool, not the last.
Push work to the engine	A Polars expression, SQL query or Spark op runs in optimized native code. A Python loop over rows does not.
Read less	Column pruning (usecols / columns=) and predicate pushdown (filters=) reduce I/O before anything reaches memory.
Right-size dtypes	int64 -> int32, float64 -> float32, object -> category. Often a 5-10x memory reduction in pandas.
Chunk and aggregate	Process in chunks, keep only running aggregates. Turns O(n) memory into O(distinct keys).
Avoid intermediate copies	Chained pandas operations each allocate. Polars lazy plans fuse operations and avoid materializing.

<code>__slots__</code>	On classes instantiated millions of times, removes the per-instance <code>__dict__</code> (roughly 40-50% smaller).
Profile before tuning	cProfile for time, memray or tracemalloc for memory, py-spy to attach to a running process.

I Profiling toolkit

```
# ---- Profile time ----
import cProfile, pstats
cProfile.run('run pipeline()', 'prof.out')
pstats.Stats('prof.out').sort_stats('cumulative').print_stats(20)

# ---- Profile memory line by line ----
# pip install memray && memray run script.py && memray flamegraph memray-*.bin
import tracemalloc
tracemalloc.start()
run pipeline()
cur, peak = tracemalloc.get_traced_memory()
print(f'current={cur/1e6:.1f} MB peak={peak/1e6:.1f} MB')

# ---- Attach to a running process without restarting it ----
# py-spy top --pid 12345
# py-spy dump --pid 12345 # see what a hung pipeline is actually doing
# ---- Constant-memory aggregation over an arbitrarily large file ----
from collections import defaultdict
totals = defaultdict(float)
for row in read_rows(path): # generator
    totals[row['customer id']] += row['amount']
# memory is O(distinct customers), not O(rows)
```

17 - PRODUCTION PIPELINE PATTERNS

I Repro structure that scales

```
orders-pipeline/
pyproject.toml # deps, entry points, tool config
README.md
.env.example # documents required config, no secrets
.pre-commit-config.yaml # ruff + mypy + tests on commit
Dockerfile
src/orders_pipeline/
__init__.py
config.py # pydantic-settings, validated at import
models.py # pydantic record schemas
extract/ # one module per source
api.py
database.py
transform/ # PURE functions: df -> df. The testable core.
clean.py
aggregate.py
load/
warehouse.py
quality/
checks.py
utils/
retrv.py
logging.py
cli.py # thin entry point, arg parsing only
daqs/
orders_daily.py # Airflow DAG imports the package, holds no logic
tests/
conftest.py
fixtures/ # small sample files
unit/
integration/
```

The single most important structural rule: **keep transformation logic in pure functions that take and return DataFrames**, with no I/O inside them. Extract and load handle I/O; transform handles logic. That separation is what makes the logic unit-testable without a cluster, a warehouse, or network access - and untestable logic is where production bugs live. The Airflow DAG should be a thin wiring layer that imports your package, never a place where business logic accumulates.

Structured Logging (not print)

I structlog

```
import structlog
log = structlog.get_logger()
log = log.bind(pipeline='orders daily', run_date='2024-06-01')
log.info('extract.start', source='api')
log.info('extract.done', rows=145 233, duration_s=42.1)
log.warning('rows.rejected', count=17, reason='null order id')
log.error('load.failed', table='silver.orders', error=str(exc))
# Emits JSON: quervable, aggregatable, alertable.
# Always log: row counts in and out, duration, and the partition processed.
# Those three fields answer 90% of 'what happened last night?' questions.
```

PART 2

45 PRACTICE QUESTIONS WITH ANSWERS

SECTION 1 - EASY (Q1 - Q15)

Core Python as it is actually used in data engineering work. These must be automatic.

Q1.

EASY

How do you read a 50 GB CSV file in Python when you only have 8 GB of RAM?

I Python Answer

```
# Option 1: generator + csv module (lowest memory, most control)
import csv, gzip
from pathlib import Path
from typing import Iterator

def read_rows(path: Path) -> Iterator[dict]:
    opener = gzip.open if path.suffix == '.gz' else open
    with opener(path, 'rt', newline='', encoding='utf-8') as fh:
        yield from csv.DictReader(fh)

totals: dict[str, float] = {}
for row in read_rows(Path('huge.csv')):
    k = row['customer_id']
    totals[k] = totals.get(k, 0.0) + float(row['amount'] or 0)

# Option 2: pandas chunks (convenient, more memory per chunk)
import pandas as pd
parts = []
for chunk in pd.read_csv('huge.csv', chunksize=1_000_000,
    usecols=['customer_id', 'amount'],
    dtype={'customer_id': 'int32', 'amount': 'float32'}):
    parts.append(chunk.groupby('customer_id')['amount'].sum())
result = pd.concat(parts).groupby(level=0).sum()

# Option 3: Polars streaming (usually the best of both)
import polars as pl
result = (pl.scan_csv('huge.csv')
    .group_by('customer_id')
    .agg(pl.col('amount').sum())
    .collect(engine='streaming'))

# Option 4: DuckDB - SQL, out-of-core, no chunking code at all
import duckdb
duckdb.sql("""SELECT customer_id, SUM(amount) FROM 'huge.csv' GROUP BY 1""").df()
```

ANSWER & EXPLANATION

The key insight is that you never need the whole file in memory - you need the whole **result**, which is usually far smaller. Four approaches, in increasing convenience: a generator with the csv module gives you constant memory and full control; pandas chunksize is convenient but each chunk still costs several times its size in RAM; Polars scan_csv with the streaming engine handles it declaratively and uses all cores; and DuckDB lets you write plain SQL against the file and spills to disk automatically. In an interview I would name the generator pattern to show I understand the mechanism, then say that in practice I would reach for Polars streaming or DuckDB because they are faster and there is less code to get wrong.

NOTE: Always mention reading fewer columns (usecols / columns=) and narrower dtypes. Halving the columns halves the problem before you write any chunking logic.

Q2.

EASY

What is a generator and why is it so important in data engineering?

I Python Answer

```
import sys

# List: everything in memory at once
```

```
squares list = [x*x for x in range(10 000 000)]
print(svs.getsizeof(squares list) / 1e6, 'MB') # ~89 MB
# Generator: one value at a time, O(1) memory
squares gen = (x*x for x in range(10 000 000))
print(svs.getsizeof(squares gen), 'bytes') # ~200 bytes
# Generator function with yield
def read_records(path):
    with open(path) as fh: # file closed when generator exhausted
        for line in fh: # file objects are already generators
            yield parse(line)
# yield from delegates to another iterable
def read_many(paths):
    for p in paths:
        yield from read_records(p)
# Composable, lazy pipeline - nothing runs until you iterate
records = read_many(['a.csv', 'b.csv'])
cleaned = (r for r in records if r['id'])
enriched = (enrich(r) for r in cleaned)
for batch in batched(enriched, 10 000):
    load(batch)
```

ANSWER & EXPLANATION

A generator produces values lazily, one at a time, holding only the current item in memory. It matters in data engineering for three reasons. **Memory**: you can process a file or stream larger than RAM with flat memory usage. **Composability**: generators chain into pipelines where each stage transforms the stream, and no intermediate stage is ever materialized - so a five-stage pipeline over 50 GB still uses constant memory. **Time to first result**: work begins immediately rather than after loading everything, which matters for streaming and for failing fast. The trade-offs to mention: a generator can only be consumed once, you cannot index it or take its length, and if you need multiple passes you must either re-create it or materialize it deliberately.

Q3.

EASY

Explain the mutable default argument trap and why it appears in pipeline code.

Python Answer

```
# BROKEN - the list is created ONCE at function definition time
def collect(record, buffer=[]):
    buffer.append(record)
    return buffer
print(collect('a')) # ['a']
print(collect('b')) # ['a', 'b'] <-- leaked across calls
# CORRECT - sentinel default, create inside the function
def collect(record, buffer=None):
    if buffer is None:
        buffer = []
    buffer.append(record)
    return buffer
# Same trap with dicts, sets, and dataclass fields
from dataclasses import dataclass, field
@dataclass
class Config:
    # sources: list[str] = [] # TypeError - dataclass catches it
    sources: list[str] = field(default_factory=list) # correct
    options: dict[str, str] = field(default_factory=dict)
```

ANSWER & EXPLANATION

Default arguments are evaluated once, when the function is defined - not on each call. So a mutable default (list, dict, set) is shared across every invocation and silently accumulates state. In pipeline code this shows up as a batch buffer or an error-collection list that grows across runs inside a long-lived worker, producing duplicated records or a slow memory leak that only manifests in production. The fix is a None sentinel with the object created inside the function body, or `field(default_factory=...)` in a dataclass. Worth noting that dataclasses actually raise a `TypeError` for mutable defaults, which is a good argument for using them over plain classes and loose dicts for pipeline config.

Q4.

EASY

What is a context manager and where do you use them in a pipeline?

I Python Answer

```
# Built-in: guaranteed cleanup even on exception
with open('data.csv') as fh: # file closed no matter what
    process(fh)

with engine.begin() as conn: # commits on success, rolls back on error
    conn.execute(insert stmt)

# Custom via contextlib - the idiomatic way
from contextlib import contextmanager
import time, tempfile, shutil
from pathlib import Path

@contextmanager
def timed(label: str):
    start = time.perf_counter()
    try:
        yield # body of the with block runs here
    finally:
        log.info('stage.done', stage=label,
            duration s=round(time.perf_counter()-start, 2))

@contextmanager
def staging_dir():
    d = Path(tempfile.mkdtemp())
    try:
        yield d
    finally:
        shutil.rmtree(d, ignore errors=True) # never leak temp files

with timed('transform'), staging_dir() as tmp:
    df.write_parquet(tmp / 'out.parquet')
upload(tmp / 'out.parquet')
```

ANSWER & EXPLANATION

A context manager guarantees setup and teardown around a block, running the cleanup even if an exception is raised. In pipelines this is essential because failures are routine: without it you leak file handles, leave database transactions open holding locks, and fill the disk with temp files that only get cleaned up when the container dies. The four places I use them constantly are file handles, database transactions (engine.begin gives commit-or-rollback semantics), temporary staging directories, and timing/instrumentation blocks. The contextlib.contextmanager decorator is the clean way to write custom ones - code before yield is setup, code in the finally block is teardown.

Q5.

EASY

How do you handle timezones and timestamps correctly in a data pipeline?

I Python Answer

```
from datetime import datetime, timezone, timedelta, date
from zoneinfo import ZoneInfo # stdlib since 3.9

# WRONG: naive datetime - no timezone, ambiguous, silently misinterpreted
bad = datetime.now()

# RIGHT: always timezone-aware, always store UTC
now_utc = datetime.now(timezone.utc)

# Parse a source timestamp that is in local time
local = datetime(2024, 3, 10, 2, 30, tzinfo=ZoneInfo('America/New York'))
stored = local.astimezone(timezone.utc) # convert for storage

# Convert back only at presentation time
display = stored.astimezone(ZoneInfo('Asia/Kolkata'))

# ISO 8601 round-trip - the safe interchange format
s = now_utc.isoformat() # '2024-06-01T14:23:11.123+00:00'
parsed = datetime.fromisoformat(s)

# Unix epoch (always UTC by definition)
epoch_s = int(now_utc.timestamp())
back = datetime.fromtimestamp(epoch_s, tz=timezone.utc)

# Partition keys: use date, not datetime, and be explicit about the tz
```

```
partition = now_utc.date().isoformat() # '2024-06-01'
```

ANSWER & EXPLANATION

The rules are simple and violating them causes some of the hardest bugs to find. **Store everything in UTC**, always as timezone-aware datetimes; never persist a naive datetime because its meaning depends on whatever machine happened to write it. **Convert to local time only at the presentation layer. Use ISO 8601 for interchange** since it is unambiguous and sortable. The specific traps: naive datetimes compared against aware ones raise a `TypeError` in Python but silently mis-join in SQL; daylight saving transitions mean some local times do not exist and others occur twice, so hour-granularity aggregation in local time can lose or duplicate an hour twice a year; and partitioning by local date means your daily partition boundary moves twice a year. Use `zoneinfo` from the standard library rather than `pytz`, which has a different and more error-prone API.

Q6.

EASY

What is the difference between a list, dict, set and tuple, and when do you use each in ETL code?

I Python Answer

```
# LIST - ordered, mutable. Batches, ordered records.
batch = []
batch.append(record) # O(1)
'x' in batch # O(n) <-- slow for lookups

# DICT - key -> value, O(1) lookup. Lookup tables, aggregation state.
lookup = {'US': 'United States', 'GB': 'Great Britain'}
country = lookup.get(code, 'Unknown') # .get avoids KeyError
from collections import defaultdict, Counter
totals = defaultdict(float)
totals[custid] += amount # no key-exists check needed
counts = Counter(r['status'] for r in records) # frequency in one line

# SET - unique, O(1) membership. Dedup, seen-keys, diffs.
seen: set[int] = set()
for r in records:
    if r['id'] in seen: # O(1) - the reason to use a set
        continue
    seen.add(r['id'])
expected = {'order_id', 'amount', 'status'}
missing = expected - set(df.columns) # set algebra for schema checks

# TUPLE - immutable, hashable. Composite keys, fixed records.
key = (customer id, order date) # valid dict key; a list is not
agg[key] = agg.get(key, 0) + amount
```

ANSWER & EXPLANATION

The choice is driven by the operation you need to be fast. **List** for ordered, mutable collections like batches - but membership testing is $O(n)$, which is the single most common accidental quadratic bug in ETL code. **Dict** for $O(1)$ keyed lookup: enrichment lookup tables and aggregation state. `defaultdict` removes the key-exists check and `Counter` gives you frequency counts in one line. **Set** for uniqueness and $O(1)$ membership: deduplication, tracking seen keys, and set algebra for schema validation (expected - actual gives you missing columns). **Tuple** for immutable, hashable values - which is why a composite key must be a tuple, not a list. The practical interview point: if you find yourself writing **if x in some_list** inside a loop over millions of records, converting that list to a set turns $O(n^2)$ into $O(n)$.

Q7.

EASY

How do you write a retry decorator with exponential backoff?

I Python Answer

```
import time, random, functools, logging
from typing import Callable, TypeVar, ParamSpec

log = logging.getLogger(__name__)
P, R = ParamSpec('P'), TypeVar('R')

def retry(
    max_attempts: int = 3,
    base_delay: float = 1.0,
    max_delay: float = 60.0,
    exceptions: tuple[type[Exception], ...] = (Exception,),
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    def decorator(fn: Callable[P, R]) -> Callable[P, R]:
        @functools.wraps(fn) # preserve name, docstring, signature
```

```
def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
    last: Exception | None = None
    for attempt in range(1, max_attempts + 1):
        try:
            return fn(*args, **kwargs)
        except exceptions as e:
            last = e
            if attempt == max_attempts:
                break
            delay = min(base_delay * 2 ** (attempt - 1), max_delay)
            delay += random.uniform(0, delay * 0.1) # jitter
            log.warning('retrying', extra={'fn': fn.name,
            'attempt': attempt, 'delay': round(delay, 2),
            'error': str(e)})
            time.sleep(delay)
            raise RuntimeError(
                f'{fn.name} failed after {max_attempts} attempts') from last
    return wrapper
return decorator

@retry(max_attempts=5, exceptions=(ConnectionError, TimeoutError))
def fetch_page(url: str) -> dict:
    return httpx.get(url, timeout=30).json()
```

ANSWER & EXPLANATION

Four details separate a correct retry decorator from a naive one. **Exponential backoff** (delay doubles each attempt) prevents you from hammering a struggling service. **Jitter** (a small random addition) prevents the thundering-herd problem where many workers all retry in lockstep. **Selective exception catching** is critical - retrying a 404 or a validation error is pointless and hides bugs, so only retry genuinely transient failures like timeouts, connection errors and 5xx responses. **functools.wraps** preserves the wrapped function's metadata, without which debugging and introspection break. I would also mention that **raise ... from last** chains the original exception so the traceback shows the real cause, and that in production I would usually use the tenacity library rather than hand-rolling this - but being able to write it demonstrates you understand what it does.

Q8.

EASY

What is the difference between shallow and deep copy, and when does it bite in data code?

I Python Answer

```
import copy

record = {'id': 1, 'tags': ['a', 'b'], 'meta': {'src': 'api'}}
# 1. Assignment - same object, not a copy at all
alias = record
alias['id'] = 99
print(record['id']) # 99 - original mutated

# 2. Shallow copy - new outer dict, SAME nested objects
shallow = record.copy() # or dict(record) / copy.copy(record)
shallow['id'] = 1 # outer change is isolated - fine
shallow['tags'].append('c') # nested change LEAKS
print(record['tags']) # ['a', 'b', 'c'] <-- bug

# 3. Deep copy - fully independent
deep = copy.deepcopy(record)
deep['tags'].append('d')
print(record['tags']) # unchanged

# In pipelines: transform functions should not mutate their input
def enrich(rec: dict) -> dict:
    out = copy.deepcopy(rec) # or build a new dict explicitly
    out['country'] = LOOKUP.get(rec['code'], 'Unknown')
    return out

# Cheaper and clearer: construct a new dict rather than deep-copying
def enrich(rec: dict) -> dict:
    return {**rec, 'country': LOOKUP.get(rec['code'], 'Unknown')}
```

ANSWER & EXPLANATION

A shallow copy creates a new outer container whose elements are still references to the same nested objects; a deep copy recursively duplicates everything. It bites in data code when a transformation function mutates a nested structure of its input -

the caller's data changes underneath them, and because the corruption happens at a distance the bug is very hard to trace. The same issue appears with pandas: many operations return views rather than copies, which is why the `SettingWithCopyWarning` exists. The clean discipline is to make transformation functions **pure**: never mutate the input, always return a new object. For flat records, constructing a new dict with unpacking is both cheaper and clearer than `deepcopy`, which is expensive on large nested structures.

Q9.

EASY

How do you validate incoming records and handle the invalid ones?

I Python Answer

```
from pydantic import BaseModel, Field, ValidationError
from datetime import date
from typing import Literal
import json

class Order(BaseModel):
    order_id: int
    customer_id: int
    amount: float = Field(gt=0)
    status: Literal['new', 'paid', 'shipped', 'cancelled']
    order_date: date

def validate_batch(raw: list[dict]) -> tuple[list[dict], list[dict]]:
    valid, rejected = [], []
    for rec in raw:
        try:
            valid.append(Order(**rec).model_dump())
        except ValidationError as e:
            rejected.append({
                'raw': json.dumps(rec),
                'errors': e.errors(), # structured, per-field
                'rejected at': datetime.now(timezone.utc).isoformat(),
            })
    return valid, rejected

valid, rejected = validate_batch(records)
load_to_silver(valid)
if rejected:
    write_dead_letter(rejected) # never lose data
    emit_metric('records.rejected', len(rejected))
    rate = len(rejected) / len(records)
    if rate > 0.01: # 1% threshold
        raise DataQualityError(f'rejection rate {rate:.1%} exceeds threshold')
```

ANSWER & EXPLANATION

Pydantic gives you declarative schema validation with type coercion and structured error reporting. The more interesting half of the question is **what you do with bad records**. Failing the entire batch on one bad row is usually wrong for ingestion - one malformed record should not block a million good ones. Silently dropping it is always wrong, because you lose data and nobody knows. The correct pattern is **quarantine**: write rejected records with their specific validation errors and a timestamp to a dead-letter table, emit a metric for the rejection count, and fail the pipeline only when the rejection **rate** crosses a threshold. That gives you three things: no data loss, the ability to reprocess after fixing the transformation, and visibility when upstream quality is degrading rather than after it has broken completely.

Q10.

EASY

What are type hints and why do they matter in production pipeline code?

I Python Answer

```
from typing import Iterator, Iterable, TypedDict, Protocol
from pathlib import Path
import polars as pl

# Function signatures document the data flow
def read_orders(path: Path) -> Iterator[dict]: ...
def clean(rows: Iterable[dict]) -> Iterator[dict]: ...
def to_frame(rows: Iterable[dict]) -> pl.DataFrame: ...
def load(df: pl.DataFrame, table: str) -> int: ... # returns rows loaded
# TypedDict: structure of a record, checked statically
```

```

class OrderRecord(TypedDict):
    order id: int
    customer id: int
    amount: float
    status: str
def total(rows: Iterable[OrderRecord]) -> float:
    return sum(r['amount'] for r in rows) # mypy catches r['amnt'] typos
# Protocol: structural typing for pluggable components
class Sink(Protocol):
    def write(self, df: pl.DataFrame) -> int: ...
def run(df: pl.DataFrame, sink: Sink) -> int:
    return sink.write(df) # any object with .write(df) -> int qualifies
# Optional / union syntax (3.10+)
def find_watermark(table: str) -> str | None: ...

```

ANSWER & EXPLANATION

Type hints are not enforced at runtime, but they matter for three practical reasons in pipeline code. **They document the data flow** - a signature reading `Iterator[dict]` to `pl.DataFrame` tells the next engineer exactly what a stage consumes and produces, which is the single most confusing thing about untyped pipeline code. **mypy catches real bugs before deployment**, particularly key-name typos in dicts (via `TypedDict`) and `None`-handling mistakes, which are the two most common runtime failures in ETL. **The IDE becomes useful** - autocomplete and inline errors instead of guessing. Protocol is worth mentioning specifically: it gives you structural typing so you can define a Sink interface that any object with the right method satisfies, without inheritance - which is how you build pluggable pipeline components cleanly.

Q11.

EASY

How do you avoid the `SettingWithCopyWarning` and chained-assignment bugs in pandas?

I Python Answer

```

import pandas as pd
# PROBLEM: chained indexing - pandas may return a view or a copy
# df[df['amount'] > 100]['status'] = 'large' # silently does nothing
# FIX 1: single .loc call - explicit, always works
df.loc[df['amount'] > 100, 'status'] = 'large'
# FIX 2: be explicit when you intend a new object
large = df[df['amount'] > 100].copy() # .copy() makes intent clear
large['status'] = 'large' # no warning, no ambiguity
# FIX 3: assign() for chainable, non-mutating transforms
out = (df
    .query('amount > 0')
    .assign(
        amount_with_tax=lambda d: d['amount'] * 1.08,
        tier=lambda d: pd.cut(d['amount'], [0,100,500,1e9],
            labels=['low','mid','high']),
    ))
# FIX 4 (pandas 2.x): copy-on-write removes the ambiguity entirely
pd.options.mode.copy_on_write = True # default in pandas 3.0

```

ANSWER & EXPLANATION

The warning appears because pandas cannot always tell whether an indexing operation returned a view into the original data or an independent copy - so an assignment through a chained index may modify the original, may modify a temporary that is immediately discarded, or may do either depending on memory layout. That non-determinism is the real danger: the code appears to work in testing and silently fails in production. The fixes are to use a single `.loc` call for conditional assignment, call `.copy()` explicitly when you intend a new independent object, and prefer `.assign()` for chained transformations because it returns a new frame and never mutates. pandas 2.x copy-on-write mode makes the semantics deterministic and becomes the default in pandas 3.0, so enabling it now is worth doing.

Q12.

EASY

How do you process files from S3 in Python?

I Python Answer

```

# Option 1: boto3 - full control, streaming
import boto3
s3 = boto3.client('s3')

```

```
# List with pagination (never assume one page)
paginator = s3.get_paginator('list_objects_v2')
keys = paginator.get_paginator('list_objects_v2')
for page in paginator.paginate(Bucket='lake', Prefix='orders/2024/')
for obj in page.get('Contents', []):
# Stream a large object without loading it into memory
body = s3.get_object(Bucket='lake', Key=keys[0])['Body']
for line in body.iter_lines():
process(line.decode())
# Option 2: s3fs / fsspec - filesystem-like, works with any library
import s3fs
fs = s3fs.S3FileSystem()
with fs.open('lake/orders/part-0.csv', 'rb') as fh:
df = pd.read_csv(fh)
# Option 3: pass the URI straight to the library (simplest)
import polars as pl
df = pl.read_parquet('s3://lake/orders/*.parquet') # globs supported
import pyarrow.dataset as ds
dataset = ds.dataset('s3://lake/orders/', partitioning='hive')
# Option 4: DuckDB - SQL directly against S3
import duckdb
duckdb.sql("INSTALL https; LOAD https;")
duckdb.sql("SELECT * FROM 's3://lake/orders/*.parquet' LIMIT 10").df()
```

ANSWER & EXPLANATION

There are four levels of abstraction and the right one depends on the task. **boto3** when you need control over the request - streaming a huge object with `iter_lines`, multipart uploads, or S3-specific features. Note the paginator: `list_objects_v2` returns at most 1000 keys per call, and assuming a single page is a classic bug that silently processes only part of the data. **s3fs/fsspec** gives a filesystem-like interface so any library that accepts a file object works unchanged, and the same code works against local disk, GCS or Azure by changing the URI scheme. **Passing the s3:// URI directly** to Polars, PyArrow or pandas is simplest and usually fastest because those libraries parallelize the reads. **DuckDB** when you want to query without an ingestion step at all. For credentials, always use IAM roles rather than embedding keys, and let boto3 resolve them from the environment.

Q13.

EASY

What is the difference between `==` and `is`, and where does it cause bugs in data code?

Python Answer

```
# == compares VALUE. is compares IDENTITY (same object in memory).
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b) # True - equal values
print(a is b) # False - different objects
# None, True, False are singletons -> always use 'is'
if value is None: # correct
    handle_missing()
if value == None: # works but wrong style; can be overridden
    handle_missing()
# The data-code trap: pandas/numpy override == to be ELEMENTWISE
import pandas as pd, numpy as np
s = pd.Series([1, None, 3])
# s == None # elementwise, all False - does NOT find nulls
s.isna() # correct way to find nulls
# np.nan is not equal to itself
print(np.nan == np.nan) # False !
print(np.nan is np.nan) # True (same object) - but do not rely on this
print(pd.isna(np.nan)) # True <- the correct check
# Ambiguous truth value: never use a Series in a boolean context
# if s: ... # ValueError
if s.notna().all(): ... # explicit reduction
```

ANSWER & EXPLANATION

`==` tests value equality and can be overridden by a class; `is` tests whether two names point at the same object in memory. For the singletons `None`, `True` and `False` you should always use `is`, both because it is faster and because a class could redefine

`__eq__` and give a surprising answer. The specific trap in data code is that pandas and NumPy override `==` to be elementwise, so `series == None` returns a Series of False rather than finding nulls - you need `.isna()`. Compounding that, NaN is not equal to itself by IEEE 754, so any equality-based null check fails. Use `pd.isna`, `.isna()` or `.notna()` and never compare to NaN directly. The related error people hit is putting a Series in an if statement, which raises 'truth value is ambiguous' - you have to reduce it explicitly with `.any()` or `.all()`.

Q14.

EASY

How do you structure logging in a data pipeline so it is actually useful?

I Python Answer

```
import structlog, logging, sys
structlog.configure(
    processors=[
        structlog.contextvars.merge_contextvars,
        structlog.processors.add_log_level,
        structlog.processors.TimeStamper(fmt='iso', utc=True),
        structlog.processors.dict_tracebacks,
        structlog.processors.JSONRenderer(), # machine-parseable
    ],
    wrapper_class=structlog.make_filtering_bound_logger(logging.INFO),
)
log = structlog.get_logger()
# Bind run-level context once - every subsequent line carries it
log = log.bind(pipeline='orders daily', run_date='2024-06-01',
run_id='a1b2c3')
log.info('extract.start', source='orders api')
log.info('extract.done', rows_in=145_233, duration_s=42.1)
log.warning('records.rejected', count=17, reason='null order id')
log.info('load.done', table='silver.orders',
rows_written=145_216, partition='2024-06-01')
try:
    risky()
except Exception:
    log.exception('load.failed', table='silver.orders') # includes traceback
raise
```

ANSWER & EXPLANATION

Structured logging emits key-value JSON rather than prose, which means logs become queryable and alertable instead of something a human greps at 3am. Three practices make the difference. **Bind run context once** - pipeline name, run date, run id - so every line is attributable to a specific run without repeating yourself. **Log the three numbers that answer most questions**: rows in, rows out, and duration, per stage. A discrepancy between rows in and rows out immediately localizes data loss. **Log the partition or window being processed**, because 'the pipeline failed' is useless without knowing which day. Beyond that: use `log.exception` inside an except block so the traceback is captured, never log secrets or PII, and use event names in a dotted namespace (`extract.done`, `load.failed`) so you can build dashboards and alerts off them.

Q15.

EASY

What is the difference between `iterrows()`, `apply()`, and vectorized operations in pandas?

I Python Answer

```
import pandas as pd, numpy as np
df = pd.DataFrame({'a': np.random.rand(1_000_000),
'b': np.random.rand(1_000_000)})
# 1. iterrows() - SLOWEST. Python loop + a Series object per row.
# out = []
# for , row in df.iterrows():
# out.append(row['a'] * row['b']) # ~60 seconds
# 2. itertuples() - faster than iterrows (namedtuples, no Series overhead)
# out = [r.a * r.b for r in df.itertuples()] # ~2 seconds
# 3. apply(axis=1) - still a per-row Python call
# df['c'] = df.apply(lambda r: r['a'] * r['b'], axis=1) # ~15 seconds
# 4. VECTORIZED - the whole column in C. ~10 milliseconds.
df['c'] = df['a'] * df['b']
```

```
# Conditional logic, vectorized:
df['flag'] = np.where(df['a'] > 0.5, 'high', 'low')
df['tier'] = pd.cut(df['a'], [0, .3, .7, 1], labels=['l','m','h'])
df['x'] = np.select([df.a > .9, df.a > .5], ['top','mid'], default='low')
# String ops are vectorized too
df['email_domain'] = df['email'].str.split('@').str[1]
```

ANSWER & EXPLANATION

The performance difference is roughly three orders of magnitude and it comes from where the loop runs. **Vectorized operations** hand the entire column to compiled C code in NumPy, so the loop is native and often SIMD-optimized. **apply(axis=1)** and **iterrows()** run a Python-level loop with per-row interpreter overhead, and **iterrows** additionally constructs a Series object for every row, which is why it is the slowest option. **itertuples** is a middle ground that avoids the Series construction. The practical rule is: if you are writing a loop over a DataFrame, there is almost always a vectorized equivalent - **np.where** for binary conditions, **np.select** or **pd.cut** for multi-way conditions, and the **.str** and **.dt** accessors for string and datetime operations. The only legitimate reason to iterate is when each row triggers an external side effect like an API call, and even then you should batch it.

SECTION 2 - MODERATE (Q16 - Q30)

Pipeline construction, performance work and validation - typical mid-level and senior data engineering questions.

Q16.

MODERATE

Write a memory-efficient function that deduplicates a 100 GB file on a composite key.

I Python Answer

```
import hashlib
from pathlib import Path
from typing import Iterator

# APPROACH 1: hash set - works if distinct keys fit in memory
def dedup_hashset(rows: Iterator[dict], keys: list[str]) -> Iterator[dict]:
    seen: set[bytes] = set()
    for r in rows:
        # Hash to 16 bytes instead of storing full key strings
        composite = ''.join(str(r[k]) for k in keys).encode()
        digest = hashlib.blake2b(composite, digest_size=16).digest()
        if digest in seen:
            continue
        seen.add(digest)
        yield r
    # Memory: ~16 bytes + set overhead per distinct key.
    # 500M distinct keys ~ 30-40 GB -> too much. Use approach 2.

# APPROACH 2: let an engine do it out-of-core (preferred)
import duckdb
duckdb.sql("""
COPY (
SELECT * FROM (
SELECT *, ROW_NUMBER() OVER (
PARTITION BY customer id, order date
ORDER BY updated at DESC) AS rn
FROM read_parquet('s3://lake/raw/*.parquet')
) WHERE rn = 1
) TO 's3://lake/dedup/' (FORMAT PARQUET, PARTITION BY (dt))
""")

# APPROACH 3: external sort / partition by key hash, then dedup per partition
def partition_by_hash(rows, n_buckets=256, outdir=Path('tmp')):
    writers = {}
    for r in rows:
        h = hash((r['customer id'], r['order date'])) % n_buckets
        writers.setdefault(h, open(outdir/f'b{h}.jsonl', 'w')).write(dumps(r))
    # Each bucket now contains ALL rows for its keys -> dedup independently
```

ANSWER & EXPLANATION

The decision hinges on whether the set of **distinct** keys fits in memory, not whether the file does. If it does, a hash set is optimal - and hashing the composite key to 16 bytes with blake2b instead of storing full key strings can cut memory by 5-10x while keeping collision probability negligible. If distinct keys do not fit, you have two options. The pragmatic one is to let DuckDB or Spark do it: ROW_NUMBER partitioned by the key with ORDER BY a tiebreaker gives you deterministic deduplication and both engines spill to disk automatically. The from-first-principles one is **hash partitioning**: route each row to one of N bucket files by hash of the key, which guarantees all rows sharing a key land in the same bucket, then deduplicate each bucket independently in memory. That is exactly how distributed engines implement it. I would also flag the design question: deterministic deduplication needs a tiebreaker column (updated_at, sequence number) - without one, which duplicate survives is arbitrary and varies between runs, which is a real bug.

Q17.

MODERATE

How do you reduce the memory footprint of a pandas DataFrame by 10x?

I Python Answer

```
import pandas as pd, numpy as np
def report(df, label=''):
    # ...
```

```

mb = df.memory_usage(deep=True).sum() / 1e6
print(f'{label}: {mb:.1f} MB')
# BEFORE: default dtypes
df = pd.read_csv('orders.csv')
report(df, 'default') # e.g. 2400 MB
# 1. Read only what you need (biggest win, costs nothing)
cols = ['order id', 'customer id', 'status', 'amount', 'order date']
# 2. Downcast numerics to the smallest safe type
dtypes = {
    'order id': 'int32', # int64 -> int32 = 50% saving
    'customer id': 'int32',
    'amount': 'float32', # check precision needs first
    'status': 'category', # low-cardinality string -> huge saving
}
df = pd.read_csv('orders.csv', usecols=cols, dtype=dtypes,
    parse_dates=['order date'])
report(df, 'optimized') # e.g. 260 MB
# 3. Automatic downcast helper
def shrink(df: pd.DataFrame) -> pd.DataFrame:
    for c in df.select_dtypes('integer'):
        df[c] = pd.to_numeric(df[c], downcast='integer')
    for c in df.select_dtypes('float'):
        df[c] = pd.to_numeric(df[c], downcast='float')
    for c in df.select_dtypes('object'):
        if df[c].nunique() / max(len(df), 1) < 0.5:
            df[c] = df[c].astype('category')
    return df
# 4. pandas 2.x Arrow backend - better strings and nullable types
df = pd.read_parquet('orders.parquet', dtype_backend='pyarrow')

```

ANSWER & EXPLANATION

The levers in order of impact. **Read fewer columns:** usecols eliminates memory before it is ever allocated, and in wide tables this alone is often the largest reduction. **Convert object-dtype strings to category:** a status column with five distinct values stored as Python objects costs roughly 60 bytes per row; as a category it costs one small integer plus a tiny dictionary, which is commonly a 95% reduction on that column. **Downcast numerics:** int64 to int32 halves it, float64 to float32 halves it - but check precision requirements first, because float32 has about 7 significant digits and that is not enough for currency at scale. **Parse dates on read** rather than storing them as strings. **Use the Arrow backend** in pandas 2.x, which stores strings compactly and gives proper nullable integer types instead of silently promoting to float when nulls appear. Measure with `memory_usage(deep=True)` - without `deep=True`, object columns report only the pointer size and you will get a misleadingly small number.

Q18.

MODERATE

Build an incremental extraction with a persisted watermark. How do you make it correct?

I Python Answer

```

from datetime import datetime, timezone, timedelta
import json
from pathlib import Path

class Watermark:
    def __init__(self, store: Path, name: str):
        self.store, self.name = store, name
    def read(self, default: datetime) -> datetime:
        if not self.store.exists():
            return default
        data = json.loads(self.store.read_text())
        v = data.get(self.name)
        return datetime.fromisoformat(v) if v else default
    def write(self, value: datetime) -> None:
        data = json.loads(self.store.read_text()) if self.store.exists() else {}
        data[self.name] = value.isoformat()
        tmp = self.store.with_suffix('.tmp')
        tmp.write_text(json.dumps(data))

```

```

tmp.replace(self.store) # atomic - no partial write
OVERLAP = timedelta(minutes=15) # guard against clock skew / open transactions
def extract incremental(conn, wm: Watermark):
    last = wm.read(default=datetime(2020,1,1,tzinfo=timezone.utc))
    since = last - OVERLAP # deliberate re-read window
    rows = conn.execute(
        'SELECT * FROM orders WHERE updated at > :since ORDER BY updated at',
        {'since': since}).fetchall()
    if not rows:
        return 0
    max seen = max(r['updated at'] for r in rows)
    load with merge(rows) # idempotent - overlap is harmless
    wm.write(max seen) # ONLY after successful load
    return len(rows)

```

ANSWER & EXPLANATION

Four things make incremental extraction correct rather than merely working. **Write the watermark only after the load succeeds**, and write it atomically (temp file plus rename) - if you advance it first and the load then fails, you have silently and permanently skipped data. **Use an overlap window**: re-read a few minutes before the last watermark, because a row committed inside a long transaction can become visible with an `updated_at` earlier than a watermark you already recorded, and clock skew between the database and your extractor makes strict greater-than unsafe. **Make the load idempotent** with a MERGE on the business key, which is what makes the overlap harmless rather than a source of duplicates. **Derive the new watermark from the data you actually retrieved** (max `updated_at` in the batch), not from the current wall clock, which would skip anything that arrives during the run. The remaining limitation to call out honestly: watermark-based extraction cannot detect hard deletes, so if the source physically deletes rows you need log-based CDC or a periodic full reconciliation.

Q19.

MODERATE

How do you fetch data from 5000 API endpoints as fast as possible without getting rate limited?

I Python Answer

```

import asyncio, httpx, random, time
from dataclasses import dataclass
@dataclass
class Result:
    url: str
    data: dict | None = None
    error: str | None = None
class RateLimiter:
    """Token bucket: smooths requests to a sustained rate."""
    def __init__(self, rate_per_sec: float, burst: int):
        self.rate, self.capacity = rate_per_sec, burst
        self.tokens = float(burst)
        self.updated = time.monotonic()
        self.lock = asyncio.Lock()
    async def acquire(self):
        async with self.lock:
            while True:
                now = time.monotonic()
                self.tokens = min(self.capacity,
                    self.tokens + (now-self.updated)*self.rate)
                self.updated = now
                if self.tokens >= 1:
                    self.tokens -= 1
                    return
            await asyncio.sleep((1 - self.tokens) / self.rate)
    async def fetch_one(client, sem, limiter, url) -> Result:
        async with sem: # cap concurrency
            for attempt in range(5):
                await limiter.acquire() # cap request RATE
            try:
                r = await client.get(url, timeout=30.0)
                if r.status_code == 429:

```

```

wait = float(r.headers.get('Retry-After', 2*attempt))
await asyncio.sleep(wait)
continue
r.raise_for_status()
return Result(url, data=r.json())
except (httpx.TimeoutException, httpx.HTTPStatusError) as e:
if attempt == 4:
return Result(url, error=str(e))
await asyncio.sleep(2*attempt + random.random())
return Result(url, error='exhausted')
async def fetch_all(urls, concurrency=20, rate=10.0):
sem, limiter = asyncio.Semaphore(concurrency), RateLimiter(rate, burst=20)
limits = httpx.Limits(max_connections=concurrency,
max_keepalive_connections=concurrency)
async with httpx.AsyncClient(limits=limits, http2=True) as client:
return await asyncio.gather(
*(fetch_one(client, sem, limiter, u) for u in urls))

```

ANSWER & EXPLANATION

The important distinction is between **concurrency** (how many requests are in flight) and **rate** (how many requests per second) - these are different limits and APIs usually enforce the second. A Semaphore caps concurrency; a token bucket caps rate and smooths bursts. You need both. Beyond that: asyncio rather than threads because this is pure I/O and thousands of coroutines cost far less than thousands of threads; a shared AsyncClient so connections are pooled and TLS handshakes are reused, which is a large win; honour the **Retry-After** header on 429 rather than guessing; exponential backoff with jitter on transient failures; and return_exceptions or an explicit Result type so one failed endpoint does not abort the other 4999. The final point I would make is that partial failure must be a first-class outcome - collect the failures, log them with their URLs, and decide based on the failure **rate** whether the run succeeded, rather than treating any single error as fatal.

Q20.

MODERATE

Compare pandas, Polars, DuckDB and PySpark. How do you choose?

ANSWER & EXPLANATION

I choose on data size relative to one machine, and on whether the work is expressible as a query. **Pandas** for small data (under a few GB), notebook exploration and the enormous ecosystem of libraries that expect a DataFrame. Its limits are real: largely single-threaded, roughly 5-10x file size in RAM, no query optimizer, and loose type handling. **Polars** is my default for single-node work up to hundreds of GB - Rust and Arrow underneath, multi-threaded by default, a genuine lazy query optimizer that does projection and predicate pushdown, a streaming engine for larger-than-RAM, and consistent null semantics. Typically 5-30x faster than pandas at a fraction of the memory. **DuckDB** when the transformation is naturally SQL, when I want to query Parquet on object storage without an ingestion step, or when I want to unit-test SQL logic locally - it is out-of-core, vectorized and parallel, and the SQL ports to the warehouse. **PySpark** only when the data genuinely exceeds one machine or the organization already runs Spark - it buys distributed shuffles and mature streaming, at the cost of JVM overhead, cluster operations and much slower iteration. The point worth making explicitly is that the crossover has moved. Hardware with hundreds of gigabytes of RAM is now routine, and Polars and DuckDB use it far more efficiently than pandas did, so a large fraction of jobs that were put on Spark five years ago run faster and cheaper on a single node today. Defaulting to Spark for tens of GB is a common and expensive mistake.

NOTE: Interviewers respond well to a concrete threshold plus the caveat: 'under ~50 GB I would reach for Polars or DuckDB first, and I would measure before assuming I need a cluster.'

Q21.

MODERATE

Write a chunked, resumable loader that handles partial failure.

```

I Python Answer
import json
from pathlib import Path
from typing import Iterator, Iterable
import structlog
log = structlog.get_logger()
class Checkpoint:
    """Records which chunks have been committed."""
    def __init__(self, path: Path):
        self.path = path
        self.done: set[str] = set()

```

```

if path.exists():
    self.done = set(ison.loads(path.read_text()))
def is_done(self, cid: str) -> bool:
    return cid in self.done
def mark(self, cid: str) -> None:
    self.done.add(cid)
tmp = self.path.with_suffix('.tmp')
tmp.write_text(ison.dumps(sorted(self.done)))
tmp.replace(self.path) # atomic
def batched(it: Iterable[dict], size: int) -> Iterator[list[dict]]:
    batch: list[dict] = []
    for x in it:
        batch.append(x)
        if len(batch) >= size:
            yield batch; batch = []
    if batch:
        yield batch
def load resumable(rows: Iterable[dict], ckpt: Checkpoint,
    chunk size: int = 50_000) -> dict:
    stats = {'chunks': 0, 'skipped': 0, 'rows': 0, 'failed': 0}
    for i, chunk in enumerate(batched(rows, chunk_size)):
        cid = f'chunk-{i:06d}'
        if ckpt.is_done(cid):
            stats['skipped'] += 1
            continue
        try:
            merge_into_target(chunk) # idempotent write
            ckpt.mark(cid) # mark ONLY after success
            stats['chunks'] += 1; stats['rows'] += len(chunk)
        except Exception as e:
            stats['failed'] += 1
            log.error('chunk.failed', chunk=cid, error=str(e))
            write_dead_letter(cid, chunk, str(e))
            if stats['failed'] > 5:
                raise RuntimeError('too many chunk failures') from e
    return stats

```

ANSWER & EXPLANATION

Five properties make a loader production-grade. **Chunking** bounds memory and bounds the blast radius of a failure to one chunk rather than the whole run. **Checkpointing** written atomically after each successful chunk makes the job resumable - a failure at 90% resumes at 90% instead of restarting. **Idempotent writes** (a MERGE on the business key) are what make resumption safe, because a chunk that succeeded but crashed before its checkpoint was written will be reprocessed harmlessly. **Dead-lettering** preserves failed chunks with their error so nothing is lost and you can reprocess after a fix. **A failure threshold** distinguishes a transient blip from a systemic problem - one bad chunk should not stop the run, but six means something is genuinely broken and continuing just wastes time and obscures the cause. Chunk ids must be deterministic across runs (derived from a stable ordering or the partition key), otherwise the checkpoint is meaningless on retry.

Q22.

MODERATE

How do you test a data pipeline properly? What do you test and what do you not?

I Python Answer

```

# conftest.py
import pytest, polars as pl, duckdb
from datetime import date
@pytest.fixture
def raw_orders() -> pl.DataFrame:
    return pl.DataFrame({
        'order id': [1, 2, 2, 3, 4], # note the duplicate
        'customer id': [10, 20, 20, 30, None], # note the null
        'amount': [100.0, 50.0, 50.0, -5.0, 75.0], # note the negative
        'status': ['paid', 'paid', 'paid', 'paid', 'cancelled'],
        'updated at': [date(2024, 1, 1)] * 5,
    })

```

```
# test transform.py - pure functions, no I/O, fast
from pipeline.transform import clean_orders, revenue_by_customer

def test_removes_duplicates(raw_orders):
    out = clean_orders(raw_orders)
    assert out['order_id'].nunique() == out.height

def test_drops_negative_amounts(raw_orders):
    out = clean_orders(raw_orders)
    assert (out['amount'] >= 0).all()

def test_drops_rows_with_null_business_key(raw_orders):
    out = clean_orders(raw_orders)
    assert out['customer_id'].null_count() == 0

def test_empty_input_is_not_an_error():
    empty = pl.DataFrame(schema=raw_schema)
    assert clean_orders(empty).height == 0

def test_output_schema_is_stable(raw_orders):
    out = clean_orders(raw_orders)
    assert out.columns == ['order_id', 'customer_id', 'amount', 'status']

# test idempotency.py - the test people forget
def test_load_twice_does_not_duplicate(tmp_path, raw_orders):
    target = tmp_path / 'out'
    load(raw_orders, target); n1 = row_count(target)
    load(raw_orders, target); n2 = row_count(target)
    assert n1 == n2
```

ANSWER & EXPLANATION

What I test: transformation logic as pure functions on small fixtures - this is fast, deterministic and catches most real bugs. Edge cases that production will definitely produce: empty input, all-null columns, duplicates, a single row, boundary values, and unexpected categories. **Idempotency** explicitly - run the load twice and assert the row count is unchanged - because that is the property most likely to be silently broken and most damaging when it is. **Schema stability**, so a refactor cannot silently drop or rename an output column that downstream consumers depend on. And SQL logic against a real engine using DuckDB in-memory, which gives genuine SQL semantics with zero infrastructure. **What I do not test in CI:** production data volumes (slow, flaky, and not what unit tests are for - volume belongs in data quality monitoring), live external APIs (mock them, or record fixtures), and the orchestrator itself. The architectural point underneath all of this is that testability is a design property: if transformation logic is mixed with I/O, none of these tests are possible. Keeping transforms as pure DataFrame-to-DataFrame functions is what makes the pipeline testable at all.

Q23.

MODERATE

Explain the GIL and how it changes your choice of parallelism for data work.

I Python Answer

```
import time
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

def cpu_bound(n: int) -> int:
    return sum(i*i for i in range(n)) # pure Python compute

def io_bound(url: str) -> int:
    return len(httpx.get(url).content) # waits on network

# CPU-bound + threads: NO speedup. GIL serializes the bytecode.
with ThreadPoolExecutor(4) as ex:
    list(ex.map(cpu_bound, [5_000_000]*4)) # ~ same as sequential

# CPU-bound + processes: real parallelism (separate interpreters)
with ProcessPoolExecutor(4) as ex:
    list(ex.map(cpu_bound, [5_000_000]*4)) # ~4x faster

# I/O-bound + threads: works well - GIL is released during I/O
with ThreadPoolExecutor(20) as ex:
    list(ex.map(io_bound, urls))

# The DE reality: the engines already parallelize below the GIL
import polars as pl
df = pl.scan_parquet('*.parquet').group_by('k').agg(pl.col('v').sum()).collect()

# Rust threads, no GIL involvement. Adding Python threads here HURTS.
```

ANSWER & EXPLANATION

The Global Interpreter Lock allows only one thread to execute Python bytecode at a time, so threads give no speedup for CPU-bound pure-Python work. It is released during I/O and inside many C extensions, which is why threads do help for network and disk waits. That gives a simple decision rule: **threads or asyncio for I/O-bound work, processes for CPU-bound Python work**. The specifically data-engineering point, and the one that distinguishes a good answer, is that **most heavy work in a data pipeline does not touch the GIL at all**. NumPy, Polars, PyArrow, DuckDB and the Spark JVM release or bypass it and parallelize internally in native code. So for a Polars aggregation or a DuckDB query, wrapping it in a thread pool adds contention and makes it slower. The GIL matters when you are doing genuine Python-level computation - custom parsing, regex over millions of strings, cryptographic hashing - and there the right answers are a process pool, or better, expressing the work as a vectorized operation so it runs in native code instead. Worth noting Python 3.13 introduced an experimental free-threaded build, which will eventually change this calculus.

Q24.

MODERATE

How do you handle schema drift in an incoming data source?

I Python Answer

```
import polars as pl
from typing import Iterable
import structlog
log = structlog.get_logger()

EXPECTED = {
    'order id': pl.Int64,
    'customer id': pl.Int64,
    'amount': pl.Float64,
    'status': pl.Utf8,
}

REQUIRED = {'order id', 'customer id', 'amount'}

def reconcile_schema(df: pl.DataFrame) -> pl.DataFrame:
    actual = set(df.columns)
    expected = set(EXPECTED)
    missing = expected - actual
    extra = actual - expected

    # 1. Missing REQUIRED column -> fail loudly, do not guess
    if missing & REQUIRED:
        raise SchemaError(f'required columns missing: {missing & REQUIRED}')

    # 2. Missing optional column -> add as null, keep going
    for col in missing - REQUIRED:
        log.warning('schema.column missing', column=col)
        df = df.with_columns(pl.lit(None).cast(EXPECTED[col]).alias(col))

    # 3. New unexpected column -> keep it, alert, do not silently drop
    if extra:
        log.warning('schema.new columns', columns=sorted(extra))
        emit_metric('schema.drift', len(extra))

    # 4. Type mismatch -> attempt safe cast, quarantine failures
    for col, want in EXPECTED.items():
        if df.schema[col] != want:
            log.warning('schema.type changed', column=col,
                        got=str(df.schema[col]), want=str(want))
            df = df.with_columns(pl.col(col).cast(want, strict=False))

    return df.select(list(EXPECTED) + sorted(extra))
```

ANSWER & EXPLANATION

Schema drift needs a policy per **kind** of change, not one blanket rule. A **missing required column** must fail loudly - guessing or defaulting a business key silently corrupts everything downstream. A **missing optional column** can be added as typed nulls with a warning so the pipeline keeps running. A **new unexpected column** should be preserved and alerted on, never silently dropped - dropping it means the data is lost and nobody learns the source changed. A **type change** gets a non-strict cast with the failures quarantined, plus an alert, because a numeric column arriving as a string usually signals an upstream problem worth investigating rather than papering over. The architectural answer that goes beyond the code: keep the Bronze layer permissive so ingestion never breaks (absorb unknown fields into a raw JSON column if needed), and enforce the strict typed schema at the Silver boundary where quality gates live. Then push upstream for a **data contract** so breaking changes fail the producer's CI rather than your pipeline at 3am. Detecting drift is damage control; contracts prevent it.

Q25.

MODERATE

Write a Polars pipeline that computes month-over-month growth per region.

I Python Answer

```
import polars as pl

pipeline = (
    pl.scan_parquet('s3://lake/silver/orders/*.parquet')
    # Filter first so pushdown reduces the scan
    .filter(pl.col('status') != 'cancelled')
    .filter(pl.col('order date') >= pl.date(2023, 1, 1))
    # Monthly grain
    .with_columns(pl.col('order date').dt.truncate('1mo').alias('month'))
    .group_by(['region', 'month'])
    .agg([
        pl.col('amount').sum().round(2).alias('revenue'),
        pl.col('order id').n_unique().alias('orders'),
        pl.col('customer id').n_unique().alias('customers'),
    ])
    .sort(['region', 'month'])
    # Window functions via over()
    .with_columns([
        pl.col('revenue').shift(1).over('region').alias('prev revenue'),
        pl.col('revenue').cum_sum().over('region').alias('cumulative revenue'),
        pl.col('revenue').rolling_mean(3).over('region').alias('rolling 3m'),
    ])
    .with_columns([
        (pl.col('revenue') - pl.col('prev revenue')).alias('abs change'),
        (((pl.col('revenue') - pl.col('prev revenue'))
         / pl.when(pl.col('prev revenue') == 0)
         .then(None)
         .otherwise(pl.col('prev revenue')) * 100)
        .round(2).alias('pct growth')),
    ])
)

print(pipeline.explain()) # inspect optimized plan
result = pipeline.collect(engine='streaming')
result.write_parquet('s3://lake/gold/regional_growth.parquet')
```

ANSWER & EXPLANATION

The Polars idioms worth highlighting. **scan_parquet** rather than **read_parquet** makes the whole thing lazy, so the optimizer can push the filters down into the file scan and prune columns - only the data actually needed is read off S3. **Filtering before aggregating** matters for the same reason. **dt.truncate('1mo')** is the clean way to derive a monthly grain. **.over('region')** is Polars' window syntax - it partitions the expression rather than the frame, so **shift**, **cum_sum** and **rolling_mean** all restart per region without a self-join. The **when/then/otherwise guard on the denominator** is the detail people miss: without it, the first month of each region divides by null and a zero-revenue month divides by zero. Finally, **explain()** before **collect()** lets you verify the pushdown actually happened, and **collect(engine='streaming')** keeps memory bounded if the input is large.

Q26.

MODERATE

How do you parallelize processing of 10,000 files efficiently?

I Python Answer

```
from concurrent.futures import ProcessPoolExecutor, as_completed
from pathlib import Path
import polars as pl, structlog
log = structlog.get_logger()

# OPTION 1 (best): let the engine do it - one call, all cores
df = pl.scan_parquet('s3://lake/raw/*.parquet').collect() # parallel reads
# or DuckDB: SELECT * FROM read_parquet('s3://lake/raw/*.parquet')

# OPTION 2: process pool when per-file work is CPU-heavy custom Python
def process_one(path: str) -> dict:
    df = pl.read_parquet(path)
    out = heavy_custom_transform(df)
    dest = path.replace('/raw/', '/clean/')
    return out, dest
```

```

out.write_parquet(dest)
return {'path': path, 'rows': out.height}

def process_all(paths: list[str], workers: int = 8) -> tuple[list, list]:
    ok, failed = [], []
    with ProcessPoolExecutor(max_workers=workers) as ex:
        futures = {ex.submit(process_one, p): p for p in paths}
        for fut in as_completed(futures):
            p = futures[fut]
            try:
                ok.append(fut.result())
            except Exception as e:
                log.error('file.failed', path=p, error=str(e))
                failed.append({'path': p, 'error': str(e)})
    return ok, failed

# Batch small files so per-task overhead does not dominate
def chunks(seq, n):
    for i in range(0, len(seq), n):
        yield seq[i:i+n]

def process_batch(paths: list[str]) -> dict:
    df = pl.concat([pl.read_parquet(p) for p in paths])
    return {'rows': transform_and_write(df)}

```

ANSWER & EXPLANATION

The first question is whether you need to parallelize in Python at all. If the work is a standard read-transform-write, **Polars or DuckDB with a glob pattern already parallelizes across all cores in native code**, and that will beat a hand-rolled process pool while being one line. Reach for a process pool only when per-file work is genuinely CPU-bound custom Python that cannot be vectorized. When you do: use processes not threads (the GIL blocks CPU-bound threads), **batch small files together** so per-task overhead and pickling do not dominate - with 10,000 tiny files, submitting 10,000 tasks is mostly overhead, whereas 200 tasks of 50 files each is efficient. Collect failures per file rather than letting one bad file abort everything, and use `as_completed` so you get results and can log progress as they finish rather than waiting for all. Two practical notes: the worker function must be top-level to be picklable, and if the files live in object storage the bottleneck is usually network I/O rather than CPU, in which case threads or asyncio are the better fit.

Q27.

MODERATE

Design a configuration system for a pipeline that runs in dev, staging and production.

I Python Answer

```

from pydantic import SecretStr, Field, field_validator
from pydantic_settings import BaseSettings, SettingsConfigDict
from typing import Literal
from functools import lru_cache

class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file='.env', env_prefix='PIPE ', case_sensitive=False,
        extra='forbid', # typo in an env var fails at startup
    )

    env: Literal['dev', 'staging', 'prod'] = 'dev'
    warehouse_dsn: SecretStr
    s3_bucket: str
    schema_name: str = 'analytics'
    batch_size: int = Field(default=10_000, ge=100, le=1_000_000)
    max_retries: int = Field(default=3, ge=0, le=10)
    dry_run: bool = False
    log_level: str = 'INFO'

    @field_validator('schema_name')
    @classmethod
    def prefix_non_prod(cls, v: str, info) -> str:
        env = info.data.get('env', 'dev')
        return v if env == 'prod' else f'{env} {v}' # dev analytics

    @property
    def is_prod(self) -> bool:
        return self.env == 'prod'

```

```
@lru cache
def get_settings() -> Settings:
    return Settings() # parsed once, cached
# Usage - typed, validated, autocompleted
cfg = get_settings()
if cfg.is prod and cfg.driv run:
    raise ValueError('driv run must be False in prod')
engine = create_engine(cfg.warehouse_dsn.get_secret_value())
```

ANSWER & EXPLANATION

Three principles. **Fail fast at startup**: a typed settings model validates everything when the process starts, so a missing or malformed variable causes an immediate, clear error instead of an `AttributeError` three hours into a run. `extra='forbid'` is worth calling out - it catches misspelled environment variables, which otherwise fail silently by falling back to defaults. **Environment via environment variables, not code branches**: no `if env == 'prod'` scattered through the codebase; the config object holds the differences and the code reads uniformly. The schema-prefixing validator is a nice guard - it makes it structurally impossible for a dev run to write to the production schema. **Secrets never in source**: `SecretStr` prevents accidental logging (its repr is masked), and the actual values come from the cloud secret manager or the orchestrator's secret backend at runtime, with a gitignored `.env` only for local development. I would also commit a `.env.example` documenting every required variable with no real values, so onboarding does not require reading the code to discover what configuration exists.

Q28.

MODERATE

How do you write data to a warehouse efficiently from Python?

I Python Answer

```
# THE HIERARCHY (slowest to fastest) - know these numbers
# row-by-row INSERT ~ 500 rows/s <- never do this
# executemany batch ~ 5,000 rows/s
# COPY / bulk from file ~ 500,000+ rows/s
# 1. Postgres: COPY from an in-memory buffer
import io, csv, psycopg
def copy_load(rows, table, columns, dsn):
    buf = io.StringIO()
    w = csv.writer(buf)
    w.writerows([r[c] for c in columns for r in rows])
    buf.seek(0)
    with psycopg.connect(dsn) as conn, conn.cursor() as cur:
        with cur.copy(f"COPY {table} ({','.join(columns)}) "
                     f"FROM STDIN WITH CSV") as cp:
            cp.write(buf.read())
# 2. Snowflake: write Parquet -> PUT to stage -> COPY INTO
df.write_parquet('/tmp/batch.parquet')
cur.execute("PUT file:///tmp/batch.parquet @%staging_orders")
cur.execute("""COPY INTO staging_orders FROM @%staging_orders
FILE_FORMAT = (TYPE = PARQUET)
MATCH BY COLUMN NAME = CASE INSENSITIVE""")
# 3. BigQuery: load from GCS (never streaming inserts for bulk)
from google.cloud import bigquery
client = bigquery.Client()
job = client.load_table_from_uri(
    'gs://bucket/batch-*.parquet', 'proj.ds.orders',
    job_config=bigquery.LoadJobConfig(
        source_format=bigquery.SourceFormat.PARQUET,
        write_disposition='WRITE_TRUNCATE'))
job.result()
# 4. Then MERGE from staging into the target = idempotent load
MERGE SQL = """
MERGE INTO silver.orders t USING staging_orders s
ON t.order_id = s.order_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
"""
```

ANSWER & EXPLANATION

The single most important thing is the **stage-then-load pattern**: write a compressed columnar file, put it in object storage, and let the warehouse ingest it in parallel with its own bulk loader. Row-by-row INSERT is roughly three orders of magnitude slower because each statement is a network round trip plus a transaction, and it is the most common cause of 'the load takes six hours' in real systems. Beyond that: **load into a staging table, then MERGE into the target**, which makes the whole operation idempotent and atomic from the consumer's point of view - they never see a half-loaded table. Use Parquet rather than CSV for the staged file so types are preserved and the payload is smaller. Wrap the staging load and the MERGE in one transaction where the engine supports it. And for a full refresh, prefer writing to a new table and doing an atomic swap or rename over TRUNCATE-then-INSERT, since the latter leaves consumers reading an empty table for the duration of the load.

Q29.

MODERATE

Write a decorator that logs execution time, row counts and failures for pipeline stages.

I Python Answer

```
import functools, time, traceback
from typing import Callable, TypeVar, ParamSpec
import polars as pl, structlog
log = structlog.get_logger()
P, R = ParamSpec('P'), TypeVar('R')

def stage(name: str) -> Callable[[Callable[P, R]], Callable[P, R]]:
    def decorator(fn: Callable[P, R]) -> Callable[P, R]:
        @functools.wraps(fn)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            rows_in = size(args[0]) if args else None
            log.info(f'{name}.start', rows_in=rows_in)
            t0 = time.perf_counter()
            try:
                result = fn(*args, **kwargs)
            except Exception as e:
                log.error(f'{name}.failed',
                        duration_s=round(time.perf_counter()-t0, 2),
                        error_type=type(e).name, error=str(e),
                        traceback=traceback.format_exc())
                emit_metric(f'stage.{name}.failures', 1)
                raise
            dur = round(time.perf_counter() - t0, 2)
            rows_out = _size(result)
            log.info(f'{name}.done', rows_in=rows_in, rows_out=rows_out,
                    dropped=(rows_in - rows_out)
                    if (rows_in and rows_out) else None,
                    duration_s=dur)
            emit_metric(f'stage.{name}.duration_s', dur)
            emit_metric(f'stage.{name}.rows_out', rows_out or 0)
            return result
        return wrapper
    return decorator

def size(obj) -> int | None:
    if isinstance(obj, pl.DataFrame): return obj.height
    if isinstance(obj, (list, tuple)): return len(obj)
    return None

@stage('clean')
def clean_orders(df: pl.DataFrame) -> pl.DataFrame: ...

@stage('aggregate')
def aggregate(df: pl.DataFrame) -> pl.DataFrame: ...
```

ANSWER & EXPLANATION

A decorator is the right place for this because instrumentation is a cross-cutting concern - you want it on every stage without polluting the business logic in each one. The details that make it genuinely useful: **log rows in and rows out and the difference**, because the dropped count is what tells you a filter is unexpectedly aggressive or a join is fanning out; **emit metrics, not just logs**, so you can alert on duration and row-count regressions rather than reading logs after the fact; **log the failure with its type and traceback before re-raising**, so the orchestrator still sees the failure but you have the diagnostic detail; and **functools.wraps** so the decorated function keeps its name and docstring, without which tracebacks and introspection become confusing. ParamSpec and TypeVar keep the signature typed through the decorator so mypy still checks

call sites. In production I would send these metrics to Prometheus or the orchestrator's metrics backend and build the pipeline dashboard off them.

Q30.

MODERATE

How do you handle a join where one side does not fit in memory?

I Python Answer

```
import polars as pl, duckdb

# OPTION 1: streaming join in Polars - lazy, spills as needed
result = (
    pl.scan_parquet('s3://lake/orders/*.parquet') # 500 GB
    .join(pl.scan_parquet('s3://lake/customers/*.parquet'), # 2 GB
          on='customer_id', how='left')
    .group_by('segment')
    .agg(pl.col('amount').sum())
    .collect(engine='streaming')
)

# OPTION 2: DuckDB - out-of-core hash join, spills to disk automatically
duckdb.sql("""
SELECT c.segment, SUM(o.amount) AS revenue
FROM read_parquet('s3://lake/orders/*.parquet') o
JOIN read_parquet('s3://lake/customers/*.parquet') c
USING (customer id)
GROUP BY 1
""").pl()

# OPTION 3: broadcast the small side manually (dict lookup)
lookup = dict(zip(small df['customer id'], small df['segment']))
for chunk in read_chunks('orders/', size=1 000 000):
    chunk = chunk.with columns(
        pl.col('customer id').replace_strict(lookup, default='unknown')
        .alias('segment'))
    aggregate into(chunk)

# OPTION 4: hash-partition BOTH sides, then join partition-by-partition
N = 64
for i in range(N):
    left = read where('orders/', f'hash(customer id) % {N} = {i}')
    right = read where('customers/', f'hash(customer id) % {N} = {i}')
    yield left.join(right, on='customer_id') # each pair fits in memory
```

ANSWER & EXPLANATION

The first question is whether **both** sides are large or just one. If one side is small - a dimension table, a lookup - the answer is a **broadcast join**: hold the small side in memory as a dict or DataFrame and stream the large side through it in chunks. That is $O(1)$ in the large side's memory and avoids any shuffle. If both sides are large, the fundamental technique is **hash partitioning**: hash the join key into N buckets on both sides, which guarantees matching keys land in the same bucket, then join each pair of buckets independently with only that pair in memory. This is precisely what Spark's shuffle hash join does, and understanding it is what the question is really testing. In practice I would reach for DuckDB or Polars streaming first, because both implement out-of-core hash joins that spill to disk automatically and will beat a hand-rolled version. I would only implement partitioning manually if I were building infrastructure or the data genuinely exceeded a single machine, at which point the answer is Spark. Worth adding: skew matters here too - if one key dominates, one bucket will not fit, and you need to salt that key or handle it separately.

SECTION 3 - ADVANCED (Q31 - Q45)

Framework design, scale, and production engineering - senior, lead and staff level questions.

Q31.

ADVANCED

Design a reusable, config-driven ETL framework in Python. What are the key abstractions?

I Python Answer

```
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import Protocol, Iterator, Any
import polars as pl, structlog

log = structlog.get_logger()

# ---- Protocols: structural interfaces, no inheritance required ----
class Source(Protocol):
    def read(self, window: 'Window') -> pl.LazyFrame: ...

class Transform(Protocol):
    def apply(self, df: pl.LazyFrame) -> pl.LazyFrame: ...

class Sink(Protocol):
    def write(self, df: pl.DataFrame, window: 'Window') -> int: ...

class Check(Protocol):
    name: str
    def run(self, df: pl.DataFrame) -> tuple[bool, str]: ...

@dataclass(frozen=True)
class Window:
    start: str
    end: str
    partition: str # the idempotency key

# ---- The pipeline is data, not code ----
@dataclass
class Pipeline:
    name: str
    source: Source
    transforms: list[Transform] = field(default_factory=list)
    sink: Sink | None = None
    checks: list[Check] = field(default_factory=list)

    def run(self, window: Window) -> dict[str, Any]:
        bloq = log.bind(pipeline=self.name, partition=window.partition)
        bloq.info('run.start')
        lf = self.source.read(window)
        for t in self.transforms:
            lf = t.apply(lf) # still lazy - one optimized plan
        df = lf.collect(engine='streaming')
        for c in self.checks: # date BEFORE writing
            ok, msg = c.run(df)
            bloq.info('check', check=c.name, passed=ok, detail=msg)
            if not ok:
                raise DataQualityError(f'{c.name}: {msg}')
        n = self.sink.write(df, window) if self.sink else 0
        bloq.info('run.done', rows=n)
        return {'rows': n, 'partition': window.partition}
```

ANSWER & EXPLANATION

Five abstractions carry the design. **Source, Transform, Sink and Check as Protocols** rather than base classes - structural typing means a new source is any object with a read method, so components are pluggable without an inheritance hierarchy and are trivially mockable in tests. **Window as an explicit, frozen value object** carrying the partition key - this is what makes every run idempotent and backfillable, because the partition being processed is an input rather than something computed from the clock. **Transforms compose lazily**: chaining LazyFrames means the whole pipeline becomes one optimized plan rather than a series of materialized intermediate results. **Checks run before the write**, so bad data never lands. **The pipeline itself is a dataclass** - configuration, not code - which is what lets you define pipelines in YAML and add the fiftieth source as a config change rather than a new module. The trade-off I would name honestly: every abstraction layer costs debuggability, so I would

only build this after three or four concrete pipelines have shown me the real shared shape. Building the framework first almost always produces the wrong abstractions.

Q32.

ADVANCED

Implement backpressure in a Python streaming pipeline.

I Python Answer

```
import asyncio, time
from dataclasses import dataclass

# Backpressure = a slow consumer must SLOW the producer,
# not accumulate an unbounded in-memory queue until OOM.

async def producer(q: asyncio.Queue, source):
    async for record in source:
        await q.put(record) # BLOCKS when the queue is full <- the mechanism
    for _ in range(N_CONSUMERS):
        await q.put(None) # sentinel per consumer

async def consumer(q: asyncio.Queue, cid: int, batch_size=1000):
    batch = []
    while True:
        item = await q.get()
        if item is None:
            if batch: await flush(batch)
            q.task_done(); break
        batch.append(item)
        if len(batch) >= batch_size:
            await flush(batch); batch = []
            q.task_done()

async def main(source):
    q = asyncio.Queue(maxsize=10_000) # BOUNDED - this is the point
    prod = asyncio.create_task(producer(q, source))
    cons = [asyncio.create_task(consumer(q, i)) for i in range(N_CONSUMERS)]
    await asyncio.gather(prod, *cons)

# Monitoring lag tells you which side is the bottleneck
async def monitor(q: asyncio.Queue):
    while True:
        depth = q.qsize()
        log.info('queue.depth', depth=depth, capacity=q.maxsize,
                saturated=depth >= q.maxsize * 0.9)
        await asyncio.sleep(10)

# Adaptive: shed or sample load when sustained saturation occurs
if q.qsize() > q.maxsize * 0.95:
    log.warning('backpressure.shedding')
    await asyncio.sleep(1) # or sample, or spill to disk
```

ANSWER & EXPLANATION

Backpressure is the property that a slow consumer propagates its slowness upstream instead of letting work pile up. Without it, an unbounded queue grows until the process is killed by the OOM reaper - and that failure is particularly bad because it loses everything in flight. **The mechanism is simply a bounded queue:** `asyncio.Queue(maxsize=N)` makes `put()` block once full, which naturally throttles the producer to the consumer's rate. Everything else is refinement: **batch on the consumer side** so writes are efficient rather than per-record; **monitor queue depth** because sustained saturation identifies the bottleneck and near-zero depth means the producer is the constraint; and for genuinely unbounded input, decide deliberately between **load shedding** (drop or sample, acceptable for metrics), **spilling to disk** (durable, slower), and **blocking** (correct when the source is replayable, like Kafka - you just consume slower and the log holds the backlog). The connection to real systems is worth stating: this is exactly what Kafka consumer lag and Spark's `maxOffsetsPerTrigger` are doing - the replayable log is the durable buffer, so blocking is safe.

Q33.

ADVANCED

How do you build a plugin architecture so new sources can be added without touching core code?

I Python Answer

```
# ---- 1. Registry via decorator (simple, explicit) ----
from typing import Protocol, Callable

class Source(Protocol):
```

```

def read(self, cfg: dict) -> 'pl.LazyFrame': ...
    REGISTRY: dict[str, type[Source]] = {}
def register(name: str) -> Callable[[type[Source]], type[Source]]:
def deco(cls: type[Source]) -> type[Source]:
if name in REGISTRY:
    raise ValueError(f'source {name!r} already registered')
    REGISTRY[name] = cls
return cls
return deco

def get_source(name: str) -> type[Source]:
if name not in REGISTRY:
    raise KeyError(f'unknown source {name!r}; have {sorted(REGISTRY)}')
return REGISTRY[name]

@register('postgres')
class PostgresSource:
def read(self, cfg): ...

@register('rest_api')
class RestApiSource:
def read(self, cfg): ...

# ---- 2. Entry points: true third-party extensibility ----
# In the plugin package's pyproject.toml:
# [project.entry-points.'mvpipeline.sources']
# salesforce = 'mv plugin.sources:SalesforceSource'

from importlib.metadata import entry_points
def load_plugins() -> None:
for ep in entry_points(group='mvpipeline.sources'):
    REGISTRY[ep.name] = ep.load() # discovered without imports

# ---- 3. Driven entirely by config ----
# pipelines/orders.yaml
# source:
# type: postgres
# dsn env: PIPE ORDERS DSN
# query: 'SELECT * FROM orders WHERE updated at > :since'
def build(cfg: dict):
    load_plugins()
return get_source(cfg['source']['type'])().read(cfg['source'])

```

ANSWER & EXPLANATION

There are two levels. A **decorator-based registry** is enough for plugins living inside your own repo: each source registers itself under a string name, config selects it by name, and core code never imports concrete sources. That inverts the dependency - core depends on the Protocol, not on implementations - which is the actual goal. For **genuinely external** plugins, Python's **entry points** mechanism lets a separately-installed package advertise itself, so another team can pip install a connector and your framework discovers it with no code change and no import in your repo at all. Details that matter in practice: fail loudly on duplicate registration rather than silently overwriting; make the error message list available names, because 'unknown source: postgres' with no hint is a miserable debugging experience; validate each plugin's config with its own Pydantic model so a bad config fails at load time rather than mid-run; and define the interface as a Protocol so plugin authors do not have to inherit from your base class. The honest caveat: this pays off at roughly five or more sources. Below that it is speculative generality and a simple if/elif is clearer.

Q34.

ADVANCED

Diagnose and fix a Python pipeline whose memory grows steadily over a long run.

I Python Answer

```

# ---- STEP 1: Confirm it is a leak, not just a big working set ----
import tracemalloc, gc, os, psutil
tracemalloc.start(25) # keep 25 frames
snap a = tracemalloc.take_snapshot()
for i in range(10):
    process batch(i)
snap b = tracemalloc.take_snapshot()
for stat in snap_b.compare_to(snap_a, 'lineno')[15]:
    print(stat) # shows growth BY LINE

```

```
def rss mb() -> float:
    return psutil.Process(os.getpid()).memory_info().rss / 1e6
# ---- STEP 2: The usual culprits ----
# (a) Unbounded accumulator - the most common by far
results = [] # grows forever
for batch in batches:
    results.append(transform(batch)) # FIX: write per batch, keep only aggregates
# (b) Unbounded cache
from functools import lru_cache
@lru_cache(maxsize=None) # FIX: maxsize=10 000
def lookup(key): ...
# (c) Closure capturing a large object
def make_handler(big_df):
    n = big_df.height # FIX: capture the small value...
    return lambda x: x / n # ...not the DataFrame
# (d) Reference cycles with __del__ - gc cannot collect them promptly
print(len(gc.garbage))
# (e) Logging handlers / connections created per iteration
# FIX: create the engine and logger ONCE, outside the loop
# ---- STEP 3: attach to a live process without restarting ----
# py-spy dump --pid 12345
# memray run --live script.py
```

ANSWER & EXPLANATION

My process is measure, localize, then fix. **Confirm it is actually a leak:** take tracemalloc snapshots between iterations and use `compare_to`, which attributes growth to specific source lines - far more useful than watching total RSS. Track RSS per batch too, because a sawtooth that returns to baseline is normal allocator behaviour, whereas a monotonic staircase is a leak. **Then check the usual causes, roughly in order of frequency.** An unbounded accumulator list is by far the most common - someone appends every batch's result and only writes at the end; the fix is to write incrementally and retain only aggregates. An `lru_cache(maxsize=None)` on a function called with unbounded distinct keys is effectively a memory leak by design. A closure that captures a large DataFrame when it only needed a scalar from it keeps the whole object alive. Reference cycles involving `__del__` defeat prompt collection. And objects created inside the loop that should be created once - engines, sessions, logging handlers - accumulate handles. **For a running production process**, `py-spy` can attach without a restart, which is invaluable when the leak takes six hours to manifest and you cannot reproduce it locally. The structural fix that prevents most of this is streaming: if each batch is written and released, there is nowhere for memory to accumulate.

Q35.

ADVANCED

Design an idempotent, backfillable Airflow pipeline. What makes it correct?

Python Answer

```
from airflow.decorators import dag, task
from airflow.exceptions import AirflowFailException, AirflowSkipException
from datetime import datetime, timedelta
import structlog

log = structlog.get_logger()

@dag(
    dag_id='orders silver',
    schedule='@daily',
    start_date=datetime(2024, 1, 1),
    catchup=True, # historical runs are first-class
    max_active_runs=3, # bounded parallel backfill
    default_args={'retries': 3,
                  'retry_delay': timedelta(minutes=5),
                  'retry_exponential_backoff': True},
)
def orders_silver():
    @task
    def resolve_window(data interval start=None, data interval end=None) -> dict:
        # The RUN defines the window. Never datetime.now().
        return {'start': data interval start.isoformat(),
                'end': data interval end.isoformat(),
                'partition': data interval start.date().isoformat()}
```

```

@task
def extract(w: dict) -> str:
    path = f"s3://raw/orders/dt={w['partition']}/"
    n = extract_range(w['start'], w['end'], path)
    if n == 0:
        raise AirflowSkipException('no source rows in window')
    return path # pass a POINTER via XCom, never data

@task
def transform_load(path: str, w: dict) -> int:
    df = read_and_transform(path)
    # Idempotent: replaces exactly this partition
    return write_replace_partition(df, 'silver.orders', w['partition'])

@task
def quality_gate(rows: int, w: dict) -> None:
    if rows == 0:
        raise AirflowFailException('wrote 0 rows')
    prev = historical_median('silver.orders', w['partition'])
    if prev and rows < prev * 0.5:
        raise AirflowFailException(f'volume anomaly: {rows} vs median {prev}')
    w = resolve_window()
    quality_gate(transform_load(extract(w), w), w)
orders_silver()

```

ANSWER & EXPLANATION

Correctness rests on four properties. **The run defines the window:** every task receives `data_interval_start` and `end` from Airflow, and no task ever calls `datetime.now()`. This is the single most important rule, because it is what makes a backfill of January 2024 actually process January 2024 rather than today. **Writes are partition-scoped and replacing** - `replaceWhere`, `INSERT OVERWRITE PARTITION`, or `MERGE` - so a retry produces the same result as a first attempt and a backfill overwrites cleanly instead of appending duplicates. **XCom carries pointers, not payloads:** a path or a row count, never a DataFrame, because XCom is stored in the metadata database and large values will degrade the entire scheduler. **Quality gates fail the run**, including a volume anomaly check against the historical median - writing zero rows or a tenth of normal volume should page someone, not silently succeed. Two more things I would add: keep DAG files free of heavy work at parse time, since the scheduler re-parses them constantly and a database call at module level will cripple it; and bound `max_active_runs` so a catchup backfill of two years does not launch hundreds of concurrent runs and overwhelm the source system.

Q36.

ADVANCED

How do you profile and optimize a slow Python data pipeline?

```

I Python Answer
# ---- 1. MEASURE where time actually goes (never guess) ----
import cProfile, pstats, io
pr = cProfile.Profile(); pr.enable()
run_pipeline()
pr.disable()
s = io.StringIO()
pstats.Stats(pr, stream=s).sort_stats('cumulative').print_stats(25)
print(s.getvalue())

# Line-level detail on the hot function
# pip install line_profiler ; @profile decorator ; kernprof -l -v script.py
# Sampling profiler - low overhead, works on a running prod process
# pv-sdv record -o flame.svg -- python pipeline.py
# pv-sdv top --pid 12345

# ---- 2. Classify the bottleneck ----
# High time in Python-level loops -> vectorize
# High time in read/parse -> read less, better format
# High time waiting on network/DB -> concurrency, batching, pooling
# High time in serialization -> Arrow, avoid pickling big objects
# Memory pressure / swapping -> stream instead of load

# ---- 3. Common fixes, biggest first ----
# (a) Read only what you need
df = pl.scan_parquet(p).select(['id', 'amount']).filter(...) # pushdown
# (b) Replace Python loops with engine expressions

```

```
# for r in rows: out.append(r['a']*r['b'])
df = df.with columns((pl.col('a') * pl.col('b')).alias('c'))
# (c) Batch the I/O
# for r in rows: cur.execute(INSERT, r) # 500 rows/s
copy load(rows, 'target', cols, dsn) # 500k rows/s
# (d) Cache expensive repeated lookups
from functools import lru cache
@lru cache(maxsize=100 000)
def geocode(postcode: str) -> tuple[float, float]: ...
```

ANSWER & EXPLANATION

The discipline is to measure first, because intuition about Python performance is usually wrong - the bottleneck is frequently in a place nobody suspected, like a repeated schema inference or a per-row database round trip buried in a helper. I use cProfile for a full picture sorted by cumulative time, line_profiler to zoom into the one hot function, and py-spy when I need to profile something already running in production without restarting it. **Then classify the bottleneck**, because the fix depends entirely on the category: Python-level loops mean vectorize or push into the engine; slow reads mean read fewer columns and rows and prefer a columnar format; network or database wait means batch and add concurrency and pool connections; heavy serialization means move to Arrow and stop pickling large objects between processes; and memory pressure means stream rather than load. In practice the three changes that most often produce order-of-magnitude wins are: replacing row-by-row inserts with a bulk COPY, replacing a Python loop with a vectorized expression, and adding column and predicate pushdown so the job stops reading data it discards. I would only consider more hardware after those, because scaling up an inefficient job pays more money for the same waste.

Q37.

ADVANCED

Implement a generic, type-safe pipeline using generics and Protocols.

```
I Python Answer
from typing import Protocol, TypeVar, Generic, Iterable, Iterator, Callable
from dataclasses import dataclass
T in = TypeVar('T in', contravariant=True)
T out = TypeVar('T out', covariant=True)
T = TypeVar('T')
U = TypeVar('U')
V = TypeVar('V')

class Step(Protocol[T in, T out]):
    def call (self, data: T in) -> T out: ...

@dataclass(frozen=True)
class Stage(Generic[T, U]):
    name: str
    fn: Callable[[T], U]
    def call (self, data: T) -> U:
        return self.fn(data)

# Composition with correct type propagation
def then(self, nxt: 'Stage[U, V]') -> 'Stage[T, V]':
    return Stage(f'{self.name}-{nxt.name}',
        lambda d: nxt(self(d)))

# Typed stages
parse: Stage[bytes, list[dict]] = Stage('parse', parse_ison)
validate: Stage[list[dict], list[dict]] = Stage('validate', validate_rows)
frame: Stage[list[dict], 'pl.DataFrame'] = Stage('frame', to_polars)
agg: Stage['pl.DataFrame', 'pl.DataFrame'] = Stage('agg', aggregate)

# mypy verifies the whole chain type-checks end to end
pipeline: Stage[bytes, 'pl.DataFrame'] = (
    parse.then(validate).then(frame).then(agg)
)
result = pipeline(raw_bytes)
# pipeline2 = parse.then(agg) # mypy ERROR: list[dict] is not DataFrame
```

ANSWER & EXPLANATION

Generics let the type checker verify that a composed pipeline is actually connectable. Stage[T, U] declares that a stage consumes T and produces U, and the **then** method's signature Stage[U, V] to Stage[T, V] encodes the composition rule - so mypy will reject a chain where one stage's output does not match the next stage's input, at development time rather than in

production. That is a genuine class of bug eliminated statically: mismatched shapes between pipeline stages. **Protocol with variance** (contravariant input, covariant output) is what makes the abstraction accept subtypes correctly, and it means implementations need no inheritance - any callable with the right signature qualifies, which keeps components loosely coupled and easy to mock. The honest assessment I would give: this is elegant and valuable in a framework or a large shared codebase where many people wire stages together. In a small pipeline it is over-engineering - the type annotations on plain functions already give most of the benefit for a fraction of the cognitive cost. I would introduce it when the number of pipelines and contributors makes accidental mis-wiring a real and recurring risk.

Q38.**ADVANCED****How do you handle secrets, PII and compliance requirements in Python pipelines?****I Python Answer**

```
# ---- 1. Secrets: never in code, never in logs ----
import boto3, json
from functools import lru_cache
from pydantic import SecretStr
@lru_cache(maxsize=32)
def get_secret(name: str) -> dict:
    sm = boto3.client('secretsmanager')
    return json.loads(sm.get_secret_value(SecretId=name)['SecretString'])
creds = get_secret('prod/warehouse')
dsn = SecretStr(creds['dsn']) # repr is masked in logs/tracebacks

# ---- 2. PII: classify, then tokenize or hash ----
import hashlib, hmac, os
PEPPER = get_secret('prod/pii-pepper')['pepper'].encode()
def pseudonymize(value: str) -> str:
    """Deterministic, keyed hash: joinable but not reversible."""
    return hmac.new(PEPPER, value.lower().strip().encode(),
        hashlib.sha256).hexdigest()
def mask_email(email: str) -> str:
    user, _, domain = email.partition('@')
    return f'{user[:2]}***@{domain}'

# ---- 3. Column-level policy, applied declaratively ----
PII_POLICY = {
    'email': 'pseudonymize',
    'phone': 'pseudonymize',
    'ssn': 'drop',
    'full_name': 'mask',
    'customer_id': 'keep',
}
def apply_policy(df, policy=PII_POLICY):
    for col, action in policy.items():
        if col not in df.columns: continue
        if action == 'drop': df = df.drop(col)
        elif action == 'pseudonymize': df = df.with columns(
            pl.col(col).map elements(pseudonymize).alias(col))
        elif action == 'mask': df = df.with columns(
            pl.col(col).map elements(mask_email).alias(col))
    return df

# ---- 4. Log redaction filter as a safety net ----
import re, logging
PATTERNS = [re.compile(r'\b[\w.%-]+@[ \w.-]+\.[A-Za-z]{2,}\b'),
    re.compile(r'\b\d{3}-\d{2}-\d{4}\b')]
class Redact(logging.Filter):
    def filter(self, record):
        for p in PATTERNS:
            record.msg = p.sub('REDACTED', str(record.msg))
        return True
```

ANSWER & EXPLANATION

Four layers. **Secrets** come from a secret manager at runtime, cached in-process, never from source or a committed file; wrapping them in `SecretStr` means an accidental log statement or traceback prints a mask rather than the credential. **PII** gets

classified per column with an explicit policy - drop, pseudonymize, mask, or keep - so handling is declarative and auditable rather than scattered through transformation code. For pseudonymization I use a **keyed HMAC with a pepper from the secret manager**, not a plain SHA-256: an unkeyed hash of an email or phone number is trivially reversible by brute force over the small realistic input space, which is a common and serious mistake. The keyed hash stays deterministic, so joins still work, while being genuinely non-reversible without the key. A **log redaction filter** is the safety net for the inevitable case where someone logs a whole record. For right-to-erasure specifically, the pattern that actually works in an immutable lake is **crypto-shredding**: encrypt each subject's PII under a per-subject key, so deletion means destroying one key and the data becomes unreadable everywhere at once - including in backups and time-travel history, which physical row deletion cannot reach. All of this depends on column-level classification and lineage; without knowing where PII flows, you cannot honestly certify compliance.

Q39.**ADVANCED****Design a data quality framework in Python with severities and observability.****I Python Answer**

```
from dataclasses import dataclass, field
from enum import Enum
from typing import Callable, Any
import polars as pl, structlog
log = structlog.get_logger()

class Severity(str, Enum):
    BLOCK = 'block' # stop the pipeline
    WARN = 'warn' # record, alert, continue

@dataclass
class CheckResult:
    name: str; passed: bool; severity: Severity
    failing_rows: int; total_rows: int; detail: str = ''

@property
def rate(self) -> float:
    return self.failing_rows / self.total_rows if self.total_rows else 0.0

@dataclass
class Check:
    name: str
    predicate: Callable[[pl.DataFrame], pl.Series] # True = row is VALID
    severity: Severity = Severity.BLOCK
    threshold: float = 0.0 # tolerated failure rate
    def run(self, df: pl.DataFrame) -> CheckResult:
        total = df.height
        if total == 0:
            return CheckResult(self.name, False, self.severity, 0, 0, 'empty')
        valid = self.predicate(df)
        failing = int((~valid).sum())
        passed = (failing / total) <= self.threshold
        return CheckResult(self.name, passed, self.severity,
                           failing, total)

CHECKS = [
    Check('order id not null', lambda d: d['order id'].is_not_null()),
    Check('order id unique',
          lambda d: pl.Series([d['order id'].n_unique() == d.height])),
    Check('amount non negative', lambda d: d['amount'] >= 0, threshold=0.001),
    Check('status known',
          lambda d: d['status'].is_in(['new', 'paid', 'shipped', 'cancelled'])),
    Check('amount plausible', lambda d: d['amount'] < 1_000_000,
          severity=Severity.WARN),
]

def qate(df: pl.DataFrame, checks=CHECKS, table: str = '') -> list[CheckResult]:
    results = [c.run(df) for c in checks]
    for r in results:
        log.info('dq.check', table=table, check=r.name, passed=r.passed,
                 failing=r.failing_rows, rate=round(r.rate, 5))
        emit_metric(f'dq.{r.name}.failing_rate', r.rate, table=table)
    blocking = [r for r in results
```

```

if not r.passed and r.severity is Severity.BLOCK1
if blocking:
raise DataQualityError(f'{table}: ' +
', '.join(f'{r.name} ({r.failing rows} rows)' for r in blocking))
return results

```

ANSWER & EXPLANATION

Four design decisions make this useful rather than ceremonial. **Severity** distinguishes checks that must halt the pipeline from those that should be recorded and alerted on - without that distinction, teams either block on trivia and get alert fatigue, or block on nothing and get no protection. **A tolerated failure rate** rather than a binary pass/fail, because in real ingestion a 0.01% malformed rate is normal and blocking on it is operationally unsustainable, while a jump to 5% is a genuine incident. **Emit a metric for every check whether it passes or fails** - this is the part most implementations miss and it is the most valuable, because it turns quality into a time series you can trend and alert on, so you see the null rate drifting from 0.1% toward 3% before it crosses any threshold. **Return the failing row count and rate, not just a boolean**, so the error message tells an on-call engineer what actually happened. I would run these as gates between layers - Bronze to Silver and Silver to Gold - so bad data is stopped at the boundary, and I would pair them with reconciliation checks against source control totals, because schema-level assertions cannot catch logic errors that produce structurally valid but wrong numbers.

Q40.

ADVANCED

How do you package and deploy a Python data pipeline to production?

```

I Python Answer
# ---- Dockerfile: multi-stage, non-root, pinned ----
FROM python:3.12-slim AS builder
RUN pip install --no-cache-dir uv
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev # reproducible from the lockfile
FROM python:3.12-slim
RUN useradd -m -u 1000 pipeline
WORKDIR /app
COPY --from=builder /app/.venv /app/.venv
COPY src/ ./src/
ENV PATH="/app/.venv/bin:$PATH" \
PYTHONUNBUFFERED=1 \
PYTHONDONTWRITEBYTECODE=1
USER pipeline
HEALTHCHECK CMD python -c 'import orders_pipeline' || exit 1
ENTRYPOINT ['run-orders']

# ---- CI (GitHub Actions) ----
# - uv sync --frozen
# - ruff check . && ruff format --check .
# - mypy src/
# - pytest --cov=src --cov-fail-under=80
# - docker build + trivy image scan
# - push to registry tagged with the git SHA (never :latest)
# - deploy: update the orchestrator image tag

# ---- CLI entry point makes it runnable and testable ----
# src/orders_pipeline/cli.py
import typer
from datetime import date
app = typer.Typer()
@app.command()
def run(partition: str = typer.Option(..., help='YYYY-MM-DD'),
dry_run: bool = False):
from .pipeline import run_pipeline
run_pipeline(date.fromisoformat(partition), dry_run=dry_run)
def main(): app()

```

ANSWER & EXPLANATION

The deployable unit should be an **immutable container image tagged with the git SHA**, built from a lockfile so the dependency tree is byte-for-byte reproducible. Tagging with **:latest** is the mistake to avoid - it makes rollback ambiguous and means two deployments of 'the same' tag can differ. A multi-stage build keeps the runtime image small by leaving build tooling

behind, and running as a non-root user is basic hardening that security review will ask about. **A CLI entry point** (typer or argparse) is worth calling out: it makes the pipeline runnable identically from a laptop, a CI job, an Airflow KubernetesPodOperator or an ECS task, with the partition passed as an explicit argument - which is exactly what makes backfills and local debugging straightforward. **CI must gate the deploy:** lint, type check, tests with a coverage floor, and an image vulnerability scan. Beyond that: PYTHONUNBUFFERED so logs stream rather than buffering until the container dies, configuration and secrets injected at runtime rather than baked into the image, and resource limits set so one pipeline cannot starve the cluster. The DAG file in the orchestrator should reference this image and pass parameters - it should contain no business logic at all.

Q41.**ADVANCED****Implement a streaming aggregation with bounded state and late-data handling.****I Python Answer**

```
from collections import defaultdict
from dataclasses import dataclass, field
from datetime import datetime, timedelta, timezone
import structlog

log = structlog.get_logger()

@dataclass
class WindowedAggregator:
    window: timedelta = timedelta(minutes=5)
    lateness: timedelta = timedelta(minutes=10) # watermark slack
    state: dict = field(default_factory=lambda: defaultdict(float))
    max event time: datetime | None = None
    late count: int = 0

    def bucket(self, ts: datetime) -> datetime:
        epoch = int(ts.timestamp())
        size = int(self.window.total_seconds())
        return datetime.fromtimestamp(epoch - epoch % size, tz=timezone.utc)

    @property
    def watermark(self) -> datetime | None:
        if self.max event time is None: return None
        return self.max event time - self.lateness

    def add(self, event: dict) -> None:
        ts = event['event time']
        self.max event time = max(self.max event time or ts, ts)

        wm = self.watermark
        if wm and ts < wm: # too late for streaming state
            self.late_count += 1
            quarantine_late(event) # NEVER drop silently
            emit_metric('stream.late_events', 1)
        return

    key = (self.bucket(ts), event['customer id'])
    self.state[key] += event['amount']

    def emit_closed(self) -> list[dict]:
        """Emit and EVICT windows the watermark has passed - bounds state."""
        wm = self.watermark
        if wm is None: return []
        closed = [k for k in self.state if k[0] + self.window <= wm]
        out = [{'window start': k[0], 'customer id': k[1],
                'amount': self.state.pop(k)} for k in closed]
        log.info('windows.emitted', count=len(out),
                state_size=len(self.state), late_total=self.late_count)
        return out
```

ANSWER & EXPLANATION

Two mechanisms do the real work. **The watermark** - max observed event time minus a lateness allowance - is what lets you decide a window is complete. **Eviction on emit** is what bounds state: once a window closes you emit its result and delete it from the state map, so memory is proportional to the number of open windows rather than to the total history. Without eviction, a long-running streaming job accumulates every key forever and eventually dies, which is the single most common streaming failure mode. The **late-data policy** is the design decision worth defending. Events arriving before the watermark cannot go into streaming state without unbounding it, but silently dropping them loses data - and in my experience that is how a pipeline quietly

under-reports revenue for months. So I quarantine them to a durable table, emit a metric, and reconcile them in a scheduled batch job. That gives low latency for the overwhelming majority and eventual correctness for the tail. Two supporting points: aggregate on **event time, not processing time**, otherwise an outage misattributes hours of events to the wrong window; and set the lateness allowance from the **measured** distribution of arrival delay rather than guessing, then alert when late volume rises because that means the assumption is breaking.

Q42.**ADVANCED****How do you build a reconciliation framework to verify pipeline correctness?****I Python Answer**

```
from dataclasses import dataclass
from decimal import Decimal
from typing import Callable
import structlog
log = structlog.get_logger()

@dataclass
class Reconciliation:
    name: str
    source_fn: Callable[[str], Decimal] # partition -> value
    target_fn: Callable[[str], Decimal]
    tolerance: Decimal = Decimal('0') # absolute or relative
    relative: bool = False

    def check(self, partition: str) -> dict:
        src = self.source_fn(partition)
        tgt = self.target_fn(partition)
        diff = abs(src - tgt)
        limit = (self.tolerance * abs(src)) if self.relative else self.tolerance
        ok = diff <= limit
        result = {'check': self.name, 'partition': partition,
                  'source': str(src), 'target': str(tgt),
                  'diff': str(diff), 'passed': ok}
        log.info('recon.result', **result)
        emit_metric(f'recon.{self.name}.diff', float(diff))
        return result

    RECONS = [
        Reconciliation('row count',
                       source_fn=lambda p: source_count(p),
                       target_fn=lambda p: warehouse_count(p)),
        Reconciliation('revenue total',
                       source_fn=lambda p: source_sum(p, 'amount'),
                       target_fn=lambda p: warehouse_sum(p, 'amount'),
                       tolerance=Decimal('0.01')), # cents of float drift
        Reconciliation('distinct customers',
                       source_fn=lambda p: source_distinct(p, 'customer id'),
                       target_fn=lambda p: warehouse_distinct(p, 'customer id')),
    ]

    def reconcile(partition: str) -> None:
        results = [r.check(partition) for r in RECONS]
        failed = [r for r in results if not r['passed']]
        if failed:
            raise ReconciliationError(
                f'{len(failed)} reconciliation(s) failed for {partition}: '
                + ', '.join(f'{f["check"]} (src={f["source"]} tgt={f["target"]})'
                            for f in failed))
```

ANSWER & EXPLANATION

Reconciliation answers a question that schema tests structurally cannot: **are the numbers right?** A pipeline can pass every not-null, uniqueness and type assertion while a join silently fans out and doubles revenue, or a filter drops a category. The only way to catch that is to compare a business-level total against the source's own figure. What to reconcile, in order of value: **row count** per partition, **sum of the primary monetary measure**, **distinct count of the main business key**, and where possible a figure the source system itself reports independently, like a daily control total from the billing system. The details that make it work in practice: use **Decimal, not float**, for monetary comparison, because float accumulation across millions of rows produces drift that will make a correct pipeline look broken. Allow a **small tolerance**, absolute or relative, so you are not paged

for a rounding artifact, but keep it tight enough to catch real errors. **Emit the difference as a metric** so you can trend it - a slowly growing gap is a different and often more interesting signal than a sudden break. And **reconcile per partition**, not just in total, so the failure message localizes the problem to a specific day.

Q43.

ADVANCED

Design the repository and code structure for a data platform maintained by 10 engineers.

```
I Python Answer
data-platform/
pyproject.toml # workspace root
uv.lock # single locked dependency set
Makefile # make test / lint / build - one obvious entry
.pre-commit-config.yaml
libs/ # SHARED, versioned, tested independently
de-core/ # Window, Source/Sink protocols, retry, logging
de-quality/ # Check framework, reconciliation
de-io/ # warehouse clients, S3 helpers, bulk loaders
pipelines/ # one directory per DOMAIN, owned by a team
orders/
pyproject.toml # depends on libs/*
src/orders/
config.py models.py
extract/ transform/ load/ quality/
cli.py
tests/
CODEOWNERS # @checkout-team
marketing/
finance/
daqs/ # thin wiring only - NO business logic
orders daily.py
infra/ # Terraform: warehouses, buckets, IAM, k8s
docs/
adr/ # architecture decision records
runbooks/ # what to do when X fails at 3am
```

ANSWER & EXPLANATION

The organizing principle is **clear ownership with shared foundations**. A monorepo gives you atomic cross-cutting changes, one dependency lockfile and consistent tooling; per-team directories with CODEOWNERS give you the autonomy and accountability that prevents a shared codebase becoming a tragedy of the commons. The structural rules I would enforce.

Shared code lives in libs/ and is versioned and tested on its own - retry logic, the quality framework, warehouse clients.

Without this, ten engineers write ten retry decorators and none of them handle 429 correctly. **Pipelines depend on libs, never**

on each other; cross-pipeline dependencies go through published tables, not shared Python, because the alternative is a

tangle where one team's refactor breaks another team's job. **DAG files contain wiring only** - a DAG that accumulates

business logic cannot be tested and cannot be run locally. **Transformations are pure functions** in transform/, with I/O

confined to extract/ and load/, which is what makes the logic unit-testable without infrastructure. And **runbooks and ADRs are part of the repo**: at ten engineers the binding constraint stops being code and becomes knowledge transfer - the person on call at 3am is often not the person who wrote the pipeline, and a runbook is what makes that survivable.

Q44.

ADVANCED

How do you handle a pipeline that must process 1 TB per day within a 2-hour window in Python?

ANSWER & EXPLANATION

First I would do the arithmetic out loud, because it frames everything: 1 TB in 2 hours is about **145 MB/s sustained**. That is genuinely modest - a single well-provisioned machine reading compressed Parquet from object storage can exceed it. So my first move is to challenge the assumption that this needs a distributed cluster at all. **Reduce the work**. Before optimizing throughput, cut volume: read only required columns (in a wide table this is often a 5-10x reduction on its own), push predicates into the file scan so filtered data is never read, and process **incrementally** - if only 5% of the data changed, process 50 GB rather than 1 TB. This step usually decides whether the rest is even necessary. **Choose the engine to match**. If it fits a large single node after pruning, Polars with the streaming engine or DuckDB will do it with full core utilization, out-of-core spilling and no cluster to operate. If it genuinely does not fit, PySpark - but then the Python code is only building a plan and the JVM does the work, so Python performance stops being the issue. **Then remove the usual bottlenecks**. Ensure the source layout is columnar, compressed with ZSTD, and split into 128-512 MB files - thousands of small files will destroy throughput before any

compute limit is reached. Parallelize the read across files. Eliminate Python-level loops and any row-wise UDF from the hot path, since a single Python UDF over a billion rows can dominate total runtime. Use bulk staged loads rather than inserts on the write side. **Finally, engineer for the window rather than the average.** A two-hour SLA means I care about the tail: chunk the work so a failure at 90 minutes resumes rather than restarts, make every write idempotent and partition-scoped, run independent partitions concurrently with bounded parallelism, and instrument throughput per stage so I know which stage will breach the window before it does. I would also load-test at 2-3x expected volume, because the honest risk in a fixed window is not the average day - it is the Monday after a long weekend.

NOTE: Doing the MB/s arithmetic and then questioning whether a cluster is needed is a strong senior signal. Candidates who immediately propose Spark for 1 TB/day are usually over-engineering.

Q45.

ADVANCED

Design a complete production Python data pipeline. Walk through every layer.

ANSWER & EXPLANATION

I would structure the answer as the pipeline itself, then the properties that make it production-grade. **Entry point.** A typer CLI taking the partition as an explicit argument, so the identical code runs from a laptop, CI, or a Kubernetes pod launched by Airflow. The orchestrator passes the window; nothing computes it from the clock. **Config.** A pydantic-settings model validated at import, with extra='forbid' so a misspelled environment variable fails immediately rather than silently defaulting. Secrets come from the cloud secret manager at runtime, wrapped in SecretStr so they cannot leak into logs. **Extract.** Incremental by a persisted watermark with a deliberate overlap window, written atomically and only after the load succeeds. Retries with exponential backoff and jitter, honouring Retry-After, and pagination as a generator so memory stays flat. Raw output lands immutable and append-only, partitioned by ingestion date, with source and batch metadata attached. **Validate.** Pydantic per record; invalid rows go to a dead-letter table with their specific errors, a metric is emitted, and the run fails only if the rejection rate exceeds a threshold - so one bad record cannot block a million good ones and nothing is ever silently dropped. **Transform.** Pure functions taking and returning LazyFrames, chained so Polars optimizes the whole plan and pushes filters and projections into the scan. No I/O inside transforms - that separation is precisely what makes them unit-testable without infrastructure. **Quality gate.** Checks with severities and tolerated failure rates run **before** the write, so bad data never lands. Every check emits a metric whether it passes or fails, which turns quality into a trendable time series rather than a binary at the moment of failure. **Load.** Stage a Parquet file to object storage, bulk load into a staging table, then MERGE into the target - idempotent, atomic from the consumer's view, and orders of magnitude faster than row inserts. Scoped to the partition so retries and backfills are safe. **Reconcile.** Compare row count and the primary monetary total against the source's own figures per partition, using Decimal, with a tight tolerance. This is the only layer that catches logic errors that pass every schema assertion. **Observe.** Structured JSON logs with run context bound once; rows in, rows out and duration per stage; metrics for duration, volume and every quality check. Alert on freshness, volume anomaly against historical median, and rejection rate. **Ship it.** Everything in Git. CI runs ruff, mypy and pytest with a coverage floor, builds an image tagged with the git SHA, and scans it. Terraform for infrastructure. A runbook for what to do when it fails at 3am. **The property I would emphasize** is that the design is optimized for the failure case, not the happy path. Idempotency, checkpointing, dead-lettering, atomic watermark writes and partition-scoped replacement all exist because retries and backfills are routine, not exceptional. That is the difference between a script that worked once and a pipeline you can leave running.

APPENDIX A - COMMON TRAPS & INTERVIEW TIPS

TRAP: `iterrows()` / `apply(axis=1)` in a hot path

Roughly 100-1000x slower than vectorized. There is almost always an engine expression that does the same thing. This is the fastest way to fail a Python DE screen.

TRAP: Mutable default arguments

def f(buf=[]): shares that list across every call. Use None and create inside, or field(default_factory=list) in a dataclass.

TRAP: `collect()` / `toPandas()` on large data

Pulls everything to the driver. This is an OOM, not a slowdown. Same for `df.collect()` in Polars on a huge lazy frame.

TRAP: Row-by-row INSERT

~500 rows/s versus ~500,000 for a staged bulk load. This is the most common cause of a slow load in real systems.

TRAP: `datetime.now()` inside a pipeline task

Silently breaks every backfill. The run's logical window must be an input, never computed from the clock.

TRAP: Advancing the watermark before the load succeeds

If the load then fails you have permanently skipped data. Write it atomically, after success, derived from the data retrieved.

TRAP: Naive datetimes

Store timezone-aware UTC. A naive datetime means whatever the writing machine assumed, which is unrecoverable later.

TRAP: `series == None` to find nulls

Elementwise comparison returns all False. Use `.isna()` / `.is_null()`. NaN is not even equal to itself.

TRAP: `x in some_list` inside a loop

O(n) per check makes the loop quadratic. Convert to a set for O(1) membership.

TRAP: `lru_cache(maxsize=None)` on unbounded keys

That is a memory leak with a decorator on it. Always bound the cache size.

TRAP: Large objects through Airflow XCom

XCom lives in the metadata database. Pass paths and row counts, never DataFrames.

TRAP: Threads for CPU-bound Python

The GIL serializes bytecode, so there is no speedup. Use processes, or better, vectorize so the work happens in native code.

TRAP: Unkeyed hash for pseudonymizing PII

A plain SHA-256 of an email or phone is brute-forceable. Use a keyed HMAC with a pepper from the secret manager.

TRAP: Silently dropping bad records

Quarantine to a dead-letter table, emit a metric, alert on the rate. Silent drops lose data for months unnoticed.

TRAP: Mixing I/O into transformation functions

Makes the logic untestable without infrastructure, which is where production bugs come from. Keep transforms pure.

How to Answer Any Python DE Question Well

State the constraint first (data size versus memory, latency, failure tolerance). Name the approach and **why**, not just the library. Mention what you would measure. Then name the trade-off you are accepting and what would make you choose differently. Interviewers are assessing engineering judgement - a candidate who says 'I would measure whether this fits in memory before choosing between Polars and Spark' outranks one who recites API surface.

APPENDIX B - LIBRARY SELECTION CHEAT SHEET

Need	Default Choice	Notes / Alternatives
DataFrame, small (< 2 GB)	pandas	Huge ecosystem. Use dtype_backend='pyarrow' in 2.x for better memory.
DataFrame, medium-large	Polars	Lazy + multi-threaded + streaming. Usually 5-30x pandas. My default.
SQL on files, out-of-core	DuckDB	Reads Parquet/CSV/JSON from S3 directly. Also excellent for unit tests.
Truly distributed	PySpark	Only when data exceeds one machine or the org already runs Spark.
Columnar interop / file I/O	PyArrow	Dataset API, zero-copy handoff, streaming ParquetWriter.
HTTP ingestion	httpx	Sync and async in one API. requests for simple sync-only cases.
JSON parsing	orjson	2-5x faster than stdlib json. Returns/accepts bytes.
Database access	SQLAlchemy 2.0	Pooling, transactions, server-side cursors. ADBC for Arrow-native reads.
Bulk load to Postgres	psycopg COPY	Orders of magnitude faster than executemany.
Orchestration	Airflow	Dagster for asset/data-aware lineage and better local dev.
Record validation	Pydantic v2	Typed models, coercion, structured errors. Rust core, fast.
DataFrame validation	pandera	Schema contracts for pandas/polars. Great Expectations for statistical suites.
Testing	pytest + DuckDB	Fixtures for small frames; DuckDB in-memory for real SQL, zero infra.
Config & secrets	pydantic-settings	Typed env config validated at startup. SecretStr to prevent log leaks.
Logging	structlog	JSON key-value logs. Bind run context once.
Retry logic	tenacity	Battle-tested backoff, jitter, exception filtering.
CLI	typer	Type-hint driven. argparse if you want zero dependencies.
Profiling	cProfile, py-spy, memray	py-spy attaches to a running process; memray for line-level memory.
Lint / format / types	ruff + mypy	ruff replaces flake8/isort/black and is dramatically faster.
Env & dependency management	uv	Very fast, lockfile-based. Poetry or pip-tools as alternatives.

Python for Data Engineering - Handbook + 45 Practice Questions | Part 1: Toolchain . Part 2: Easy (1-15) .
Moderate (16-30) . Advanced (31-45)

Stream, do not load . Push work to the engine . Design for the retry . Measure before you tune